ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Universidade Vigo

Deep learning

Master's Degree in Artificial Intelligence

2025/26

Unit 5A – Transformers

Index

1 Introduction

2 Transformer blocks

3 Encoder-only transfomers

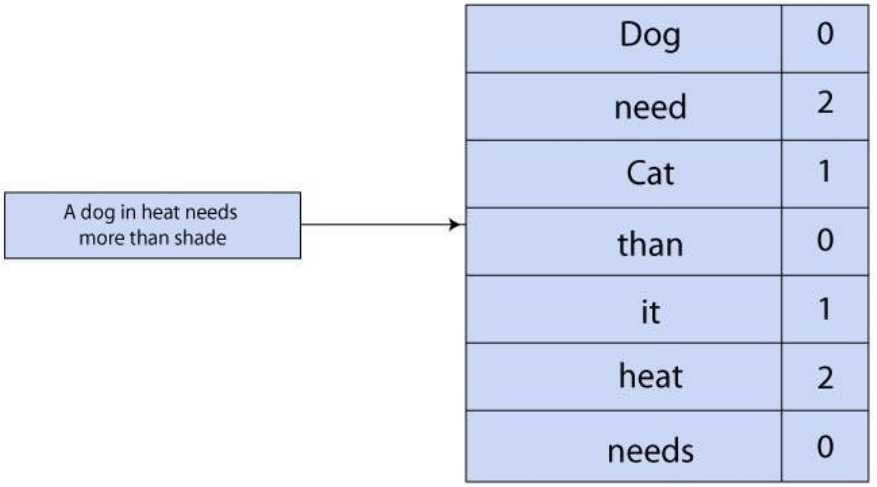
4 Decoder-only transformers

5 Encoder-decoder architecture

6 Examples of transformers

1 Introduction

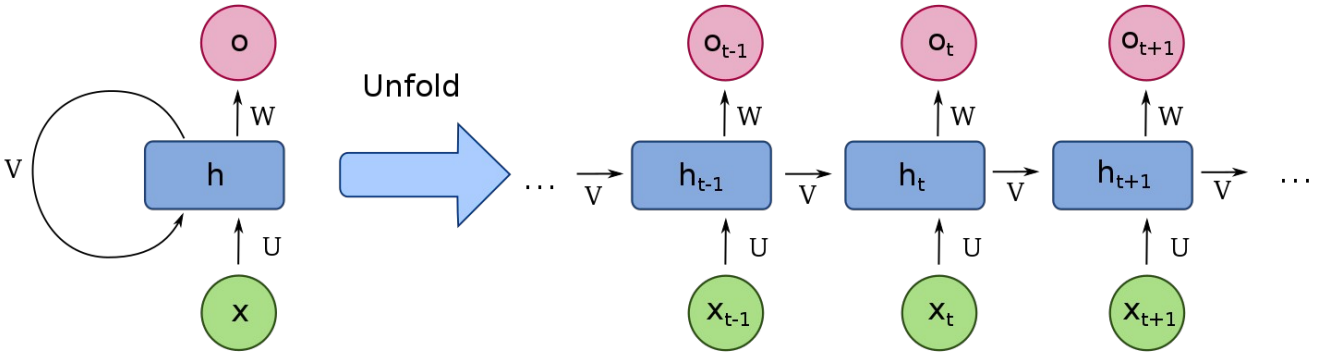
Earliest approaches in NLP (Natural Language Processing): *bags of words* and *n-grams*



Uni-Gram	This	Is	Big	Data	AI	Book
Bi-Gram	This is	Is Big	Big Data	Data AI	AI Book	
Tri-Gram	This is Big	Is Big Data	Big Data AI	Data AI Book		

Very crude and limited for complex NLP applications

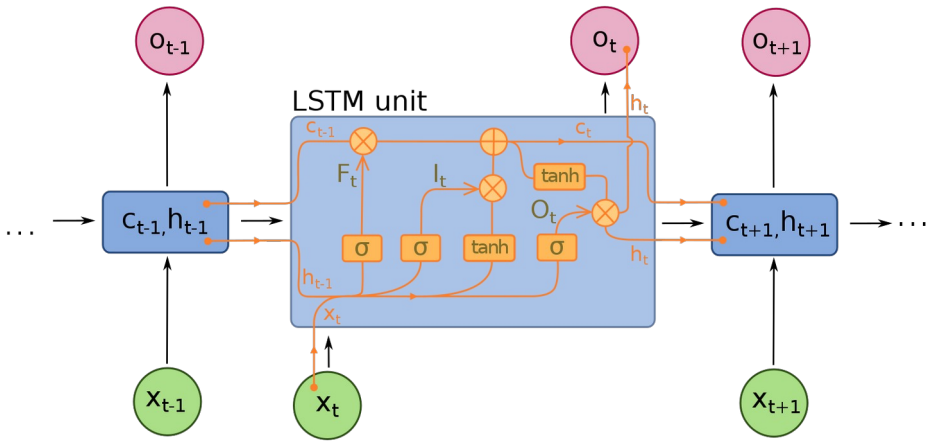
RNN (Recurrent Neural Networks): loops for information persistence

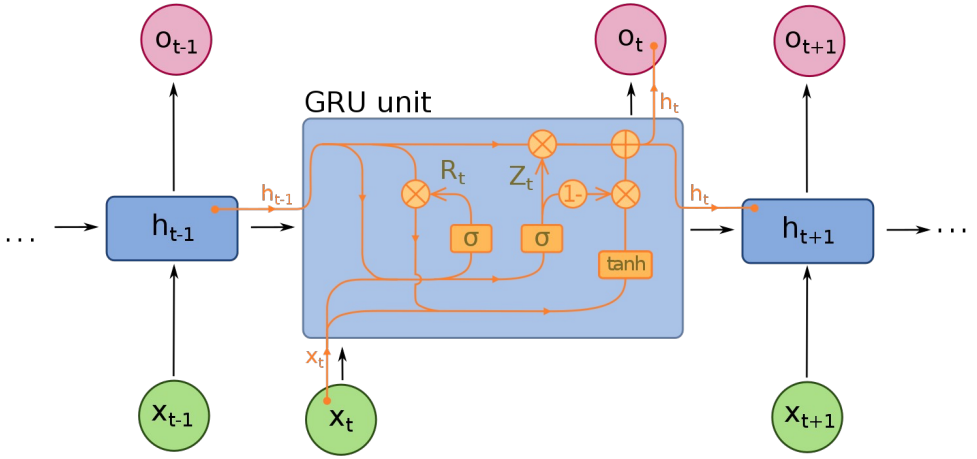


Problems with RNNs:

- Valid only for short-term dependencies
- Vanishing gradients

LSTM (Long Short-Term Memory) and *GRU (Gated Recurrent Unit)* networks: the problems of RNNs are reduced, but they are still present

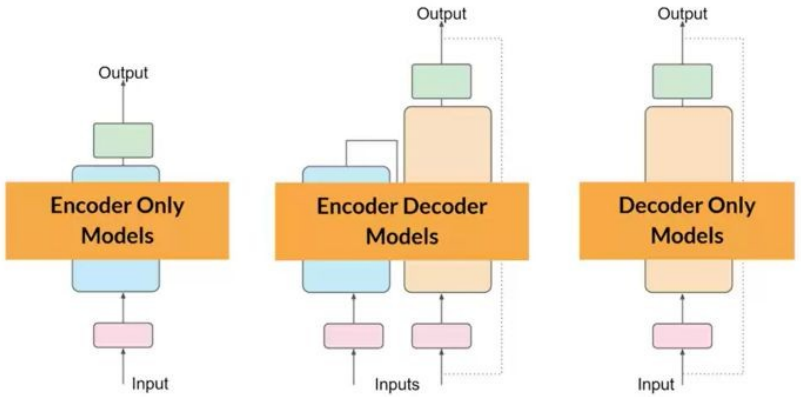


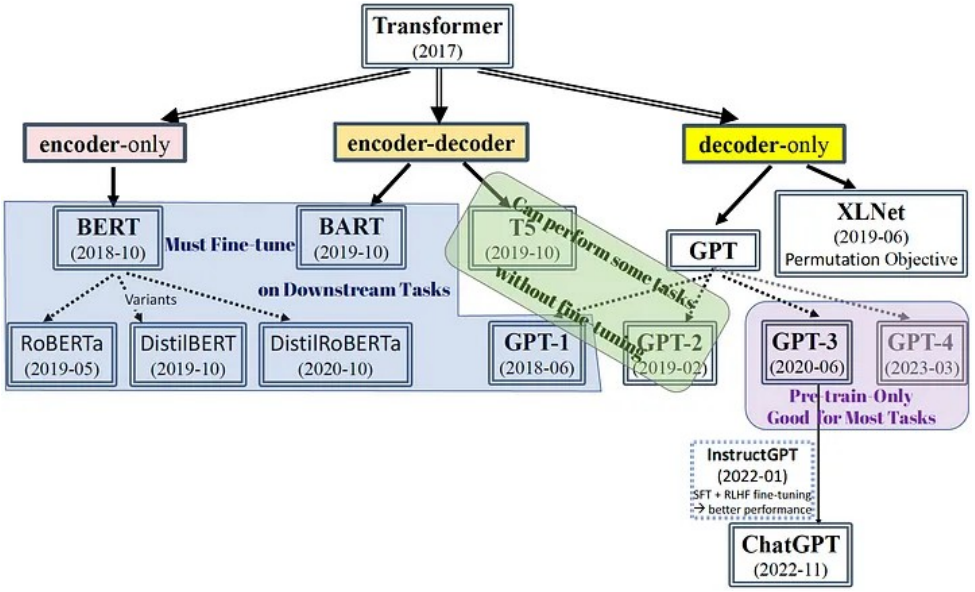


"Attention is All You Need", Vaswani, et al., 2017 proposed the **Transformer architecture** to address the issue of long-term dependencies, using:

- **Attention mechanisms**
- **Positional encodings**

The proposed architecture has an **encoder** (for text input) and a **decoder** (for generating text).





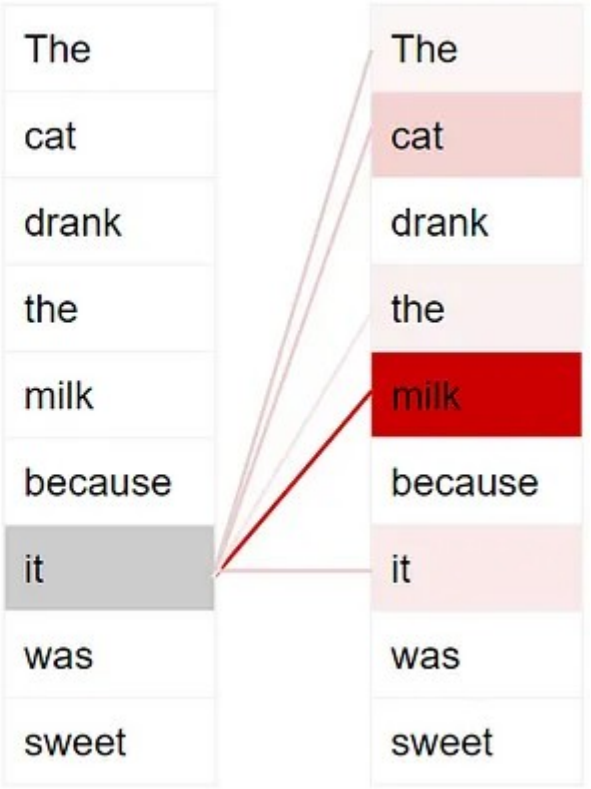
- **Encoder-only**: ***understands text***. Ex: text classification, document embedding, NER (Named Entity Recognition), Text Summarization
- **Decoder-only**: ***generates text*.** Ex: GPT
- **Encoder-decoder**: ***transforms text*.** Ex: machine translation

2 Transformer blocks

Let's suppose that a model is processing a word: **attention** is a mechanism that enables the model to focus on other words related to that word

Example: we are writing a text

The cat drank the milk because it was ...



Queries, Keys and Values

Example:

The pink elephant tried to get into the car but it was too ...

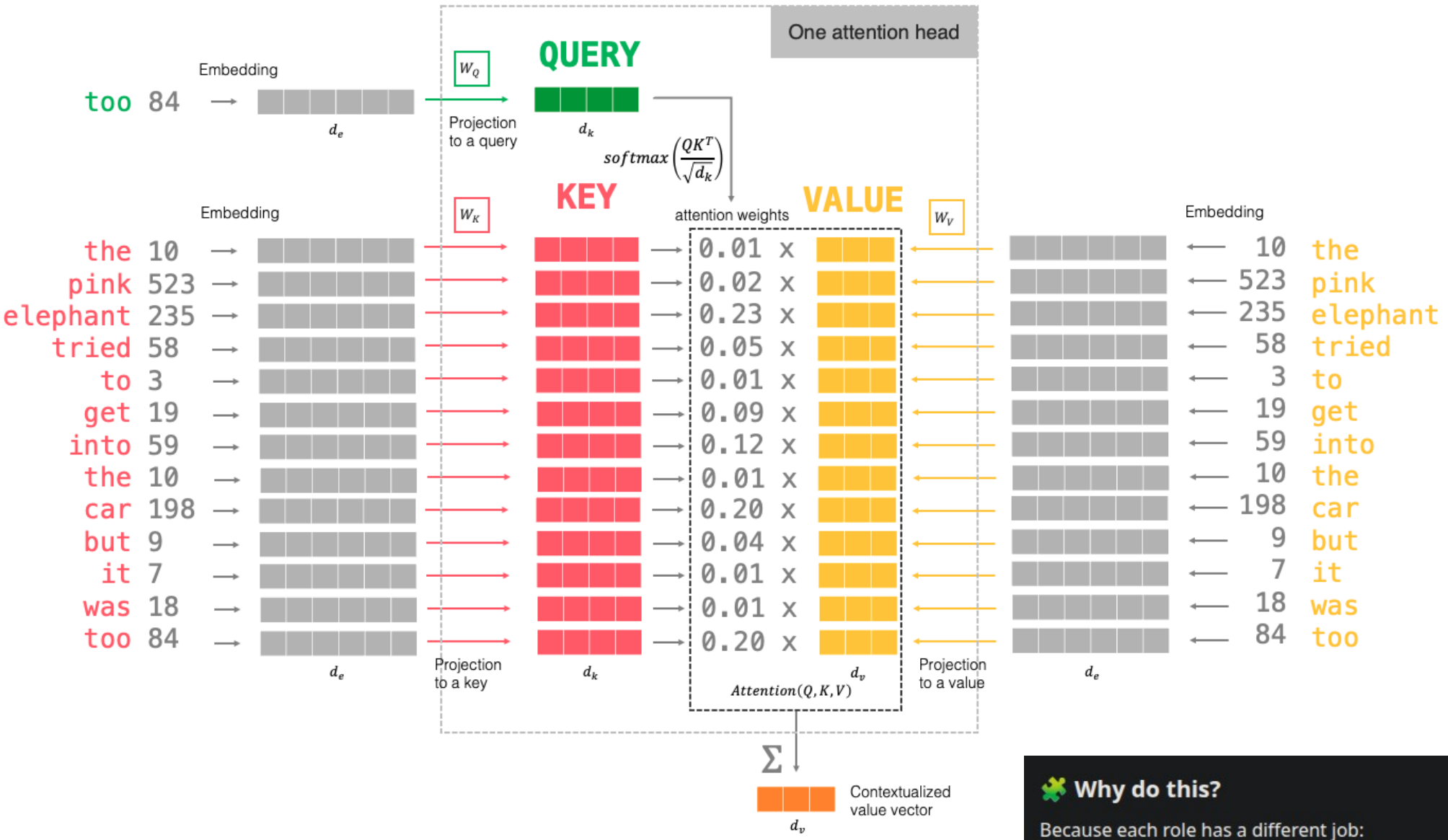
We want to predict the next word. Each word tries to contribute to this decision, but this decision is going to be weighted:

- The word **elephant** contributes a lot to complete with a word related to size
- The word **pink** contributes very little

Attention head: implements the attention mechanism with:

- **Query** (*what word follows too?*): the information asked about the input sentence
- **Key**: the information in the input that the attention mechanism evaluates against the query
- **Value**: content or information associated with each position in the input

The **attention weights** are calculated from the query and key, and they are applied to the values



🧩 Why do this?

Because each role has a different job:

- **Query (Q)** → "What am I looking for?"
- **Key (K)** → "What do I contain?"
- **Value (V)** → "What information do I give?"

This is part of the Attention Mechanism.

- The **query** (Q) is a representation of the current task, obtained from the embedding of the word *too* passed through a matrix W_Q (dimension changes from d_e to d_k)

- The **key** vectors (K) represent each embedded word passed through a matrix W_k. Dimension of these vectors is also d_k

- Each key is compared against the query with dot product (QK^T). The higher this number, the more the word and the query resonate (and they contribute more to the attention head)

- These numbers are scaled by $\sqrt{(d_k)}$ and softmax is applied. This is a vector of **attention weights.** They represent the importance or relevance assigned to different parts of the input sequence, to focus on relevant information and to capture dependencies between positions effectively.

- The **value vectors** (V) represent each word in the sentence. Obtained by passing embedding of words through a matrix W_v (dimension changes from d_e to d_v). *Often* $d_v = d_k$

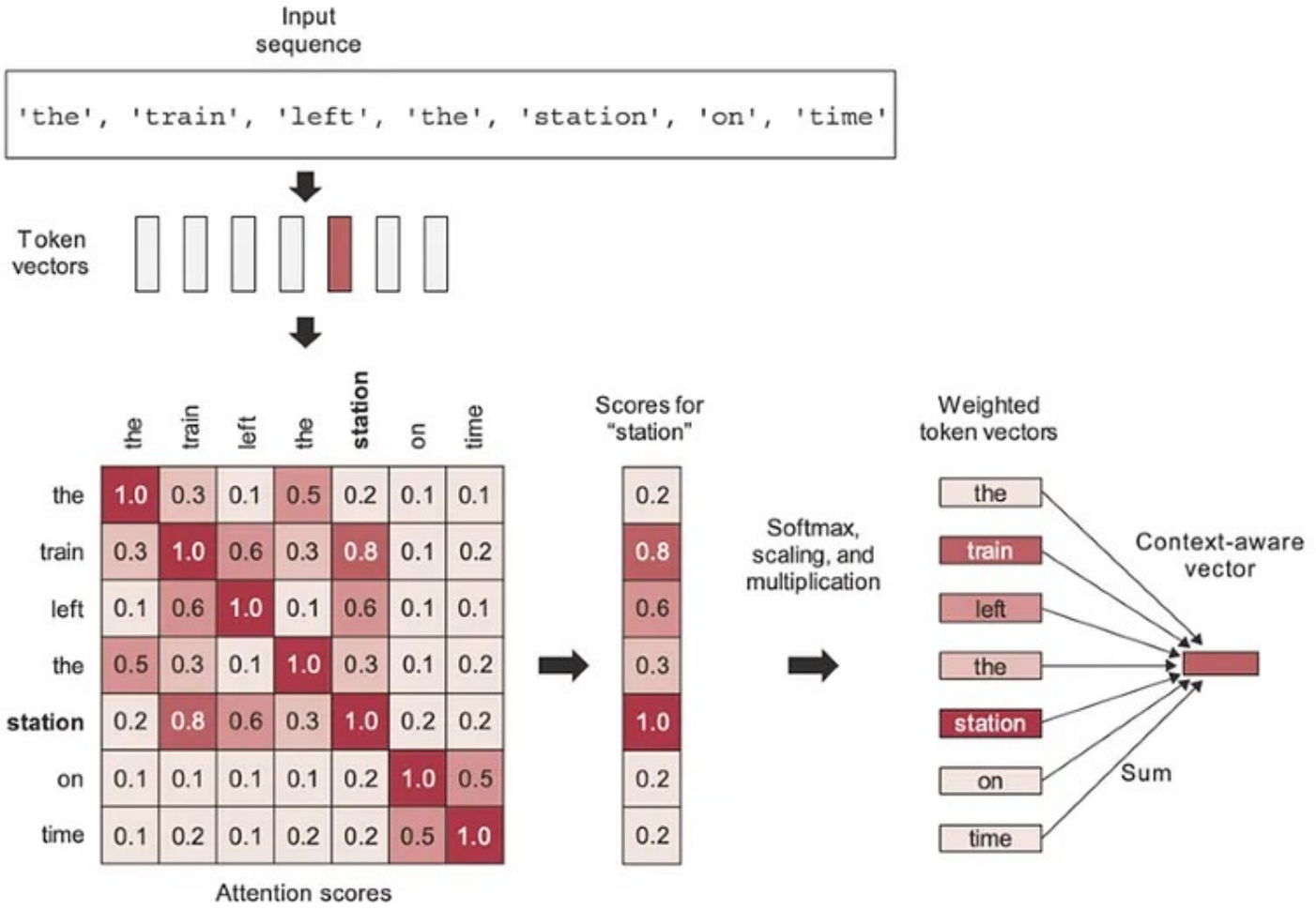
- Value vectors are multiplied by the attention weights to obtain the **attention**

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- The attention is summed to give a vector of length d_v. This **value vector** contains a blended opinion from words in the sentence on the task of predicting what word follows *too*. **It is passed along to the next layer or used for making predictions**

- **Matrices W_Q, W_k *and* W_v are trained**

Self-attention: if we calculate all the relationships in a sentence, we will have a square matrix (a vector per word)



Positional encoding: there is nothing that cares about the order of the words. This is problem, because we need the attention layer to predict different outputs for these sentences

The dog looked at the boy and ... (barked?)

The boy looked at the dog and ... (smiled?)

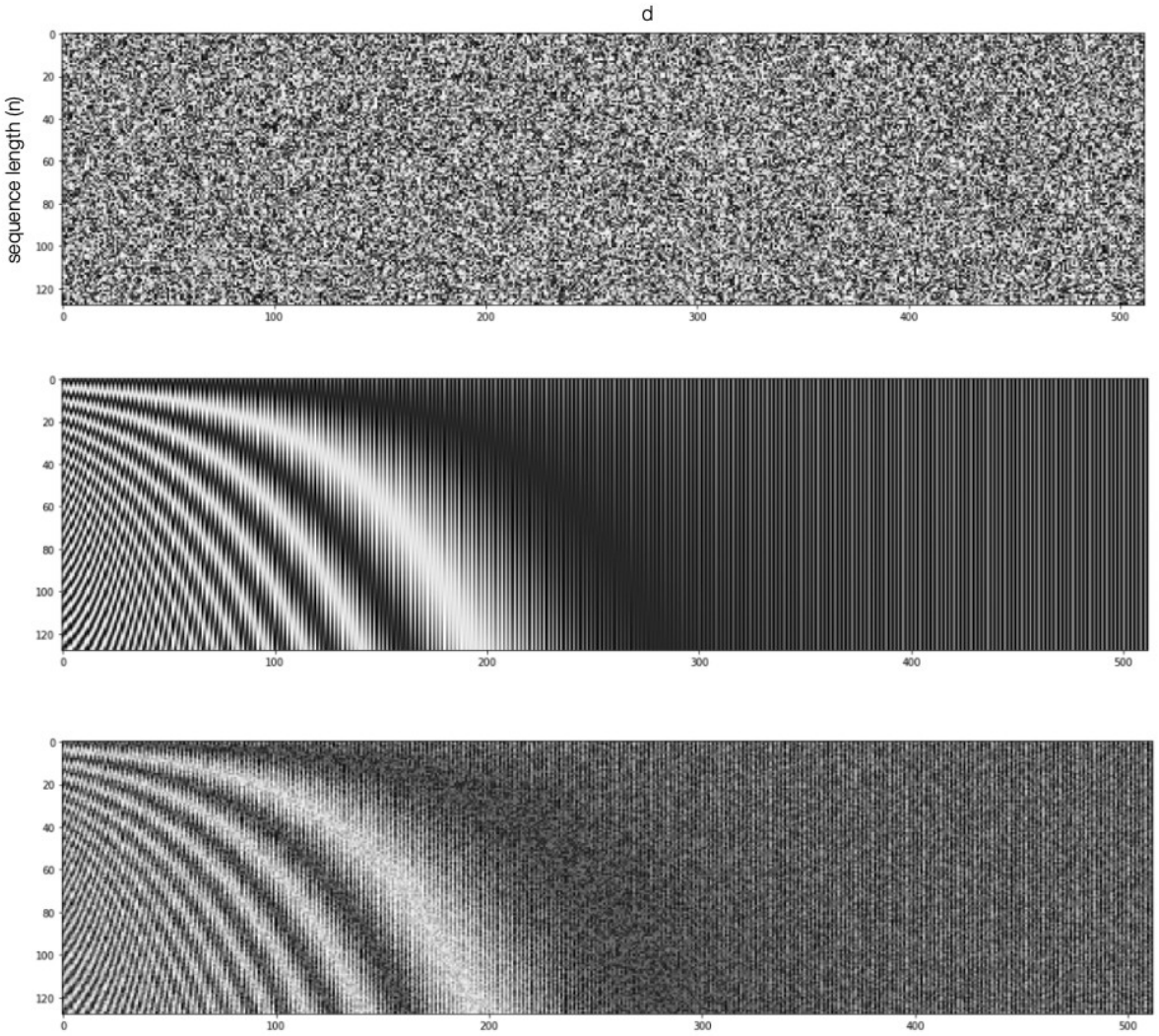
For each word, we combine:

- Tokenization
- Positional embedding

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

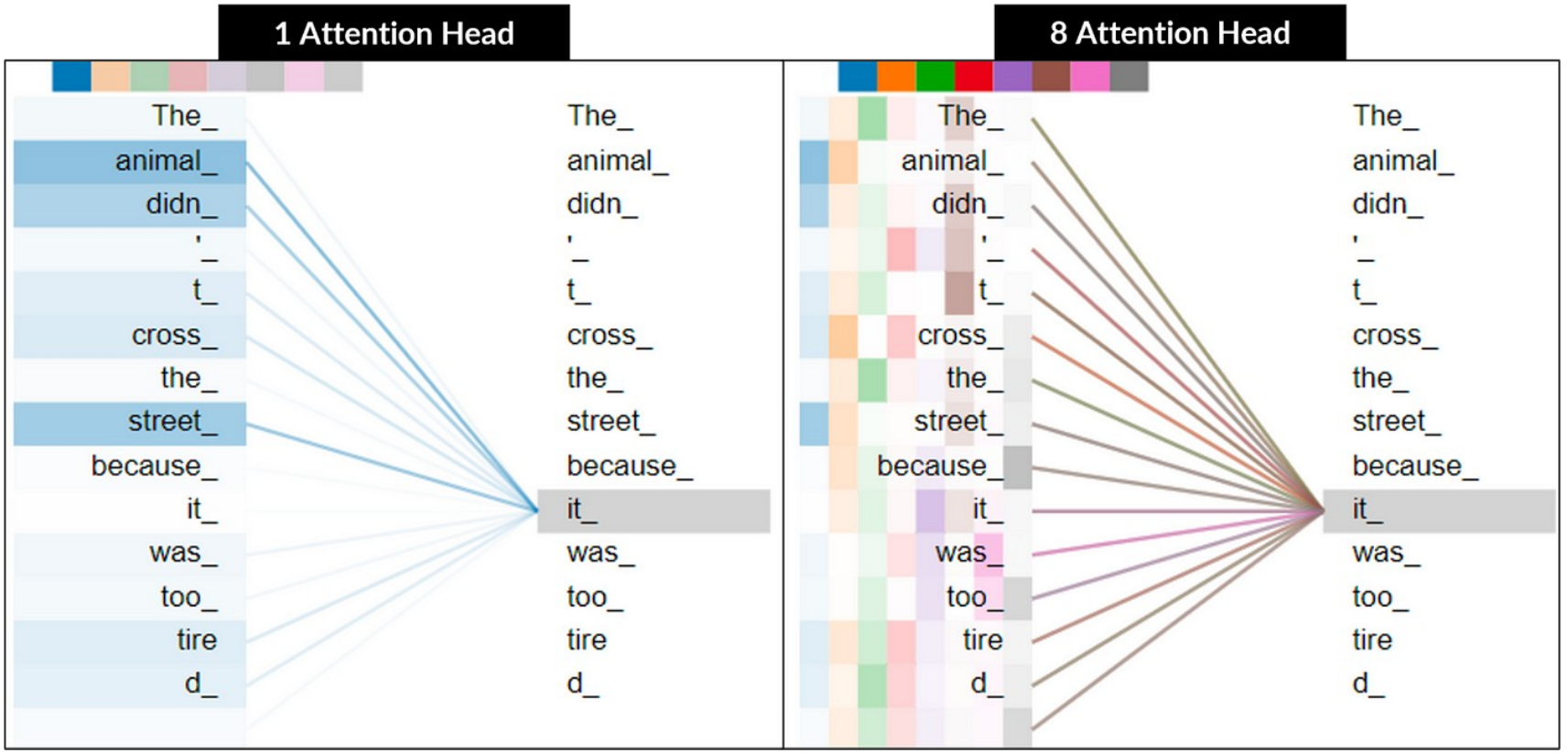
where

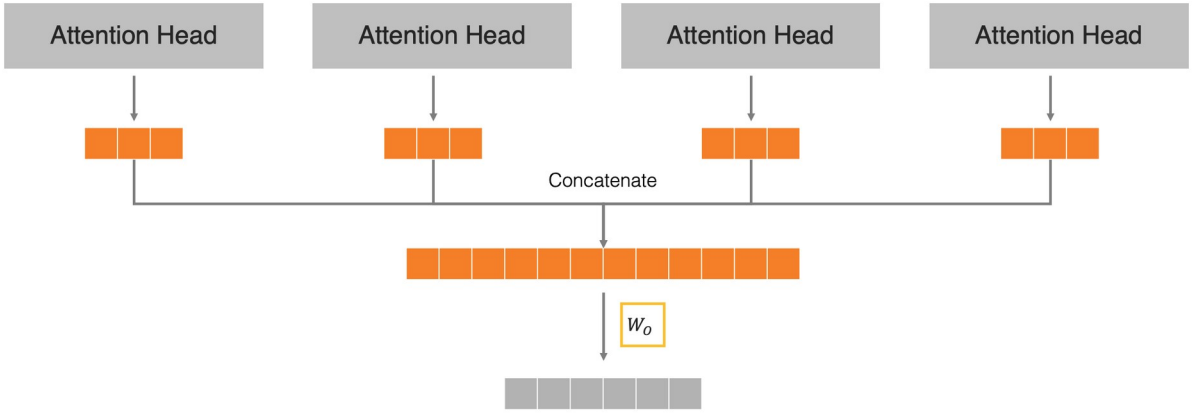
- *pos* is word position
- d_{model} is the length of the encoding vector
- *i* is the index value

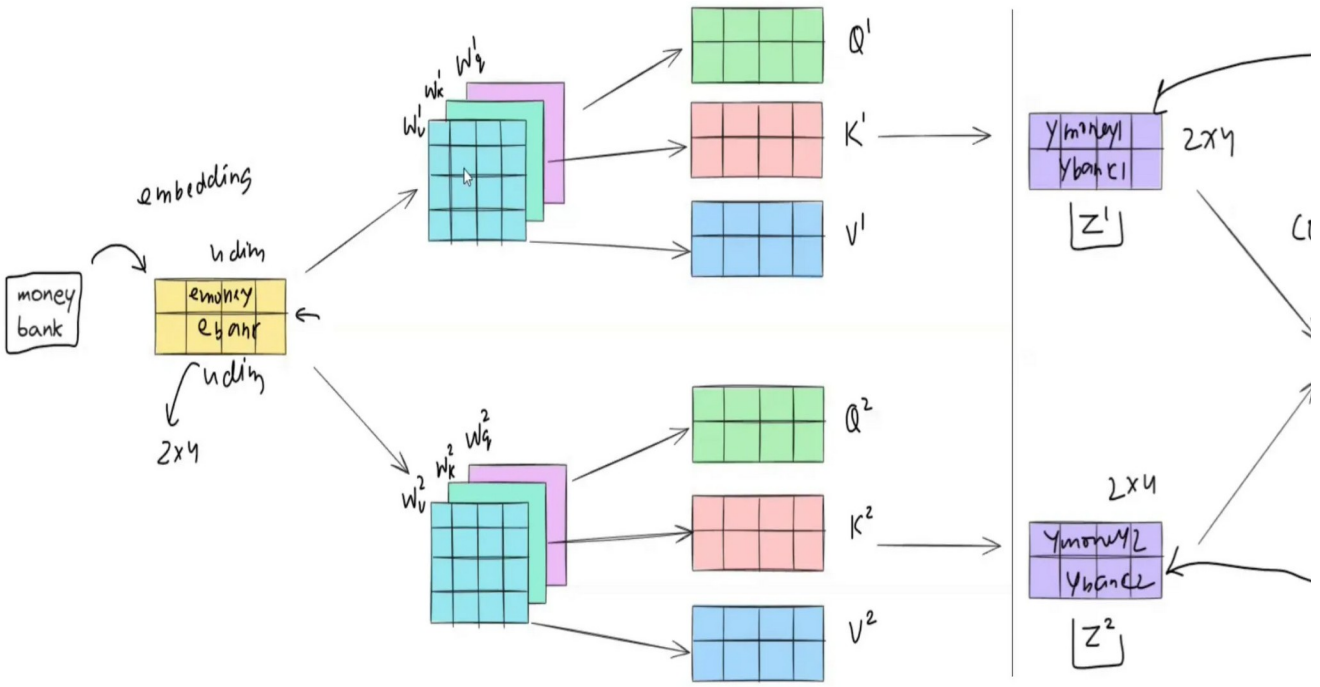


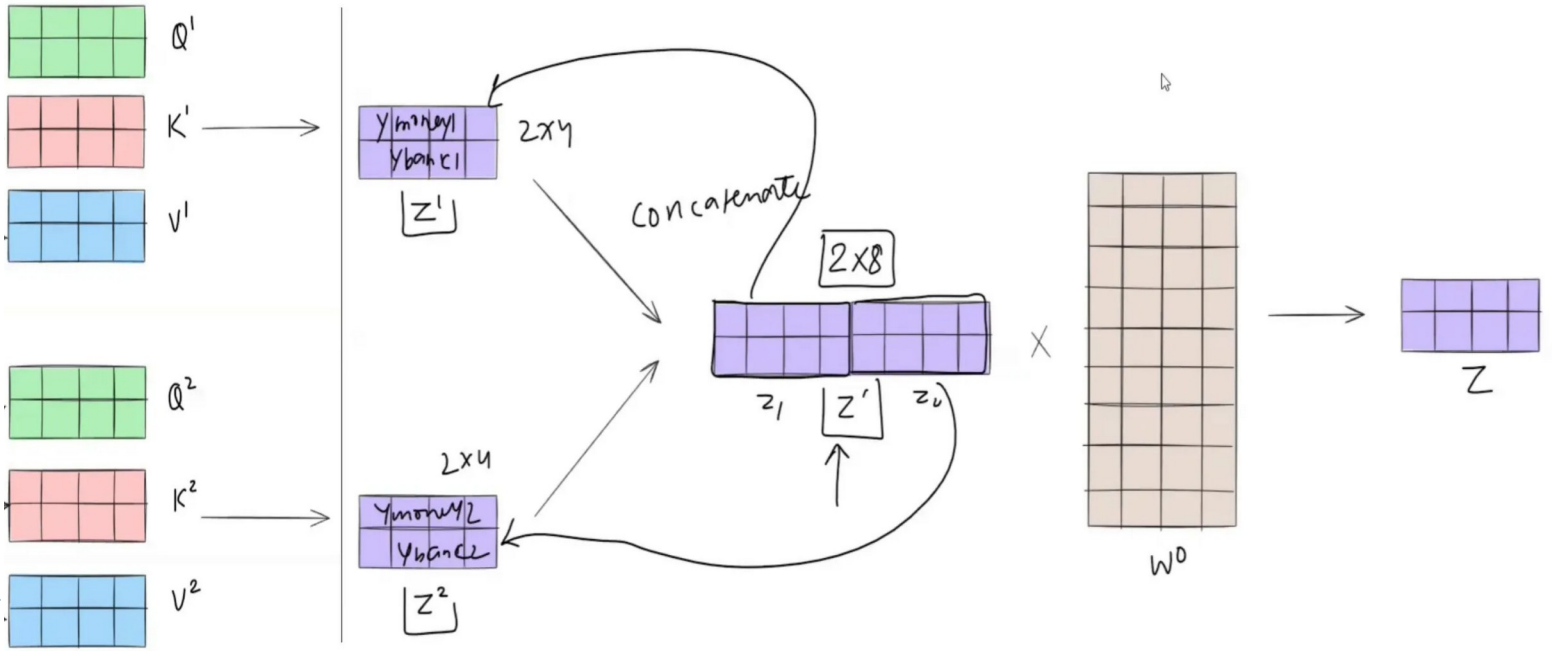
Each row corresponds to a word's encoding

Multi-head attention: several attention heads can be used in parallel, allowing to learn different attentions so more complex relationships can be learnt






embedding
4 dim
money
bank
money
bank
4 dim
2x4
Q1
K1
V1
Q2
K2
V2
2x4
2x4
Z1
Z2


Q1
K1
V1
Q2
K2
V2
2x4
2x4
Z1
Z2
concatenate
2x8
z1
z'
zv
W0
Z

Masking

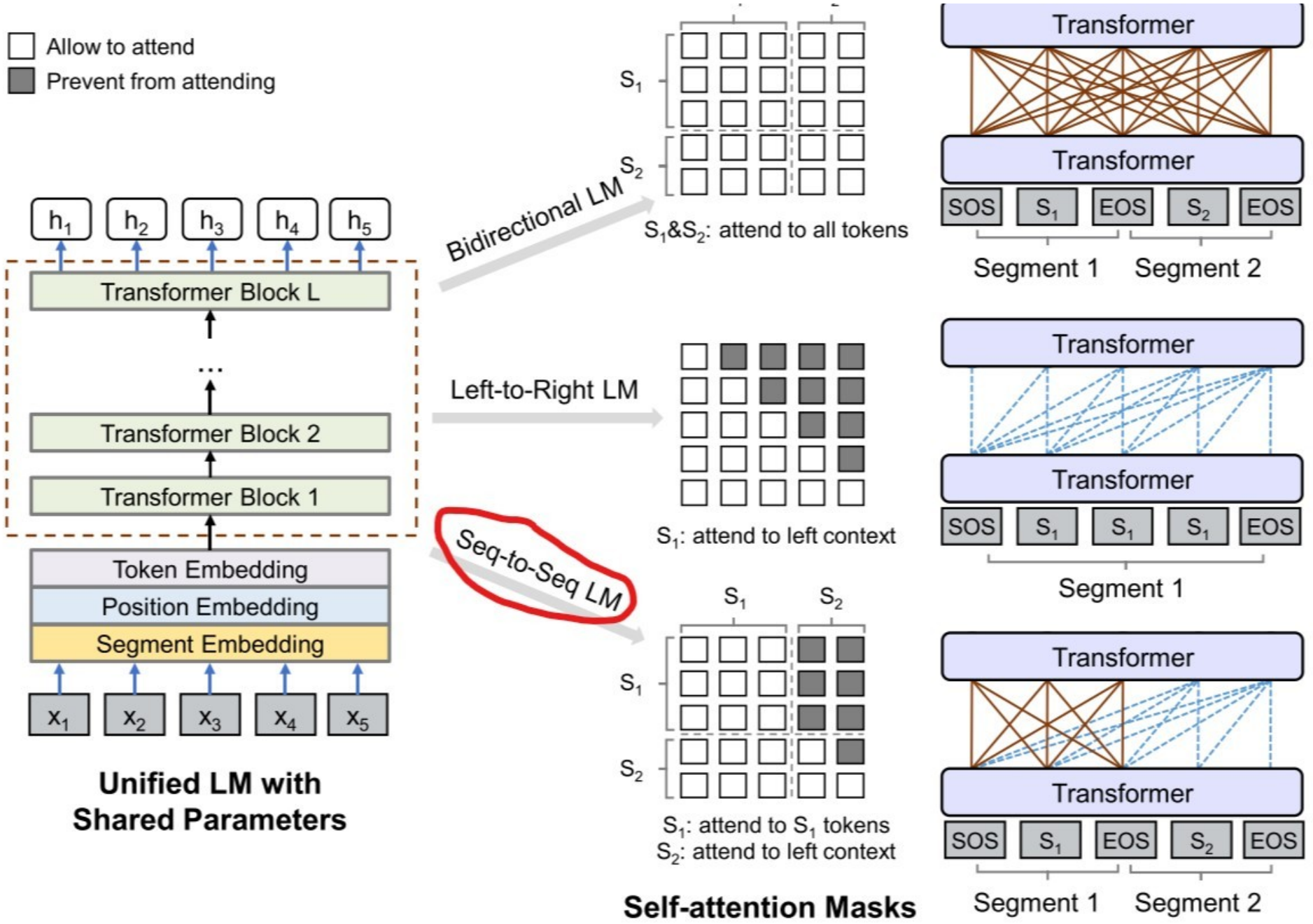
Depending on the application, attention may be restricted from right to left (words only depending on the other words that precede it).

For example, GPT (Generative Pre-Trained) models: given a sentence, their output is the next word in the sentence.

When we train a GPT model, we have to use **causal masking** to avoid that the model peeks into the future. In attention mechanisms, **causal masking is applied just before the softmax step in the computation of attention scores**.

Example: for predicting the fourth word, the query would be '*river*', and the attention can be only on the first three words '*on the river*'.

	On	the	river	bank
On	0.7	0	0	0
the	0.2	0.9	0	0
river	0.4	0.1	0.8	0
bank	0.3	0.4	0.3	0.8


Allow to attend
Prevent from attending
h1 h2 h3 h4 h5
Transformer Block L
...
Transformer Block 2
Transformer Block 1
Token Embedding
Position Embedding
Segment Embedding
x1 x2 x3 x4 x5
Unified LM with Shared Parameters
Bidirectional LM
S1
S2
S1&S2: attend to all tokens
Transformer
Transformer
SOS S1 EOS S2 EOS
Segment 1 Segment 2
Left-to-Right LM
S1: attend to left context
Transformer
Transformer
SOS S1 S1 S1 EOS
Segment 1
Seq-to-Seq LM
S1 S2
S1
S2
S1: attend to S1 tokens
S2: attend to left context
Transformer
Transformer
SOS S1 EOS S2 EOS
Segment 1 Segment 2
Self-attention Masks

A **Transformer block** is a component within a Transformer that applies some skip connections, feed-forward (dense) layers and normalisation around the multi-head attention layer

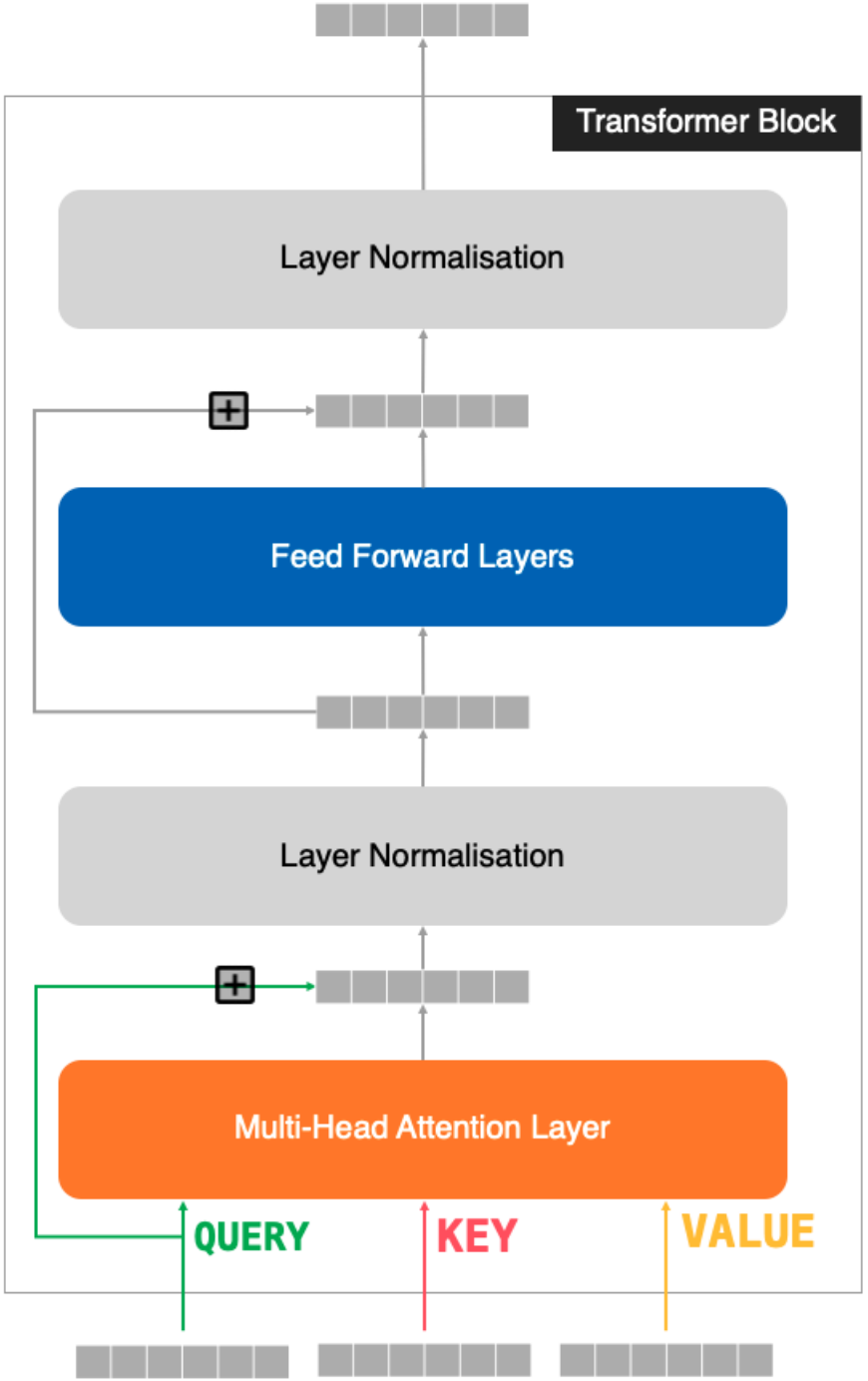
- *Skip connections* to avoid vanishing gradients
- *Layer normalization* to provide stability
- *Feed forward layer* (dense layer) to allow the extraction of high level features

The inputs to the transfomer block are tensors

(batch_size, sequence_len, embed_dim)

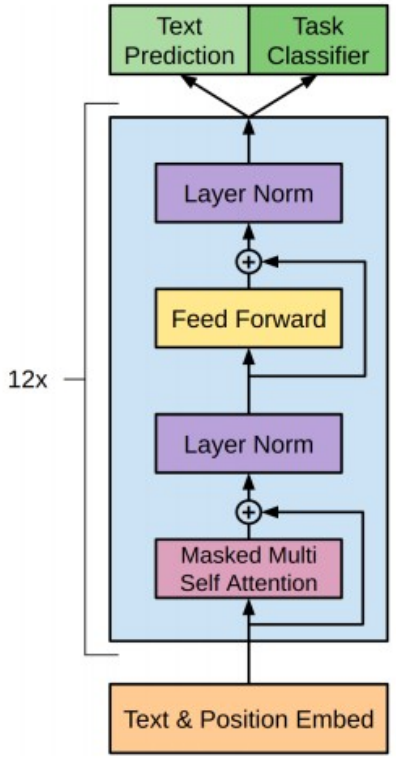
The output of the transfomer block is a tensor

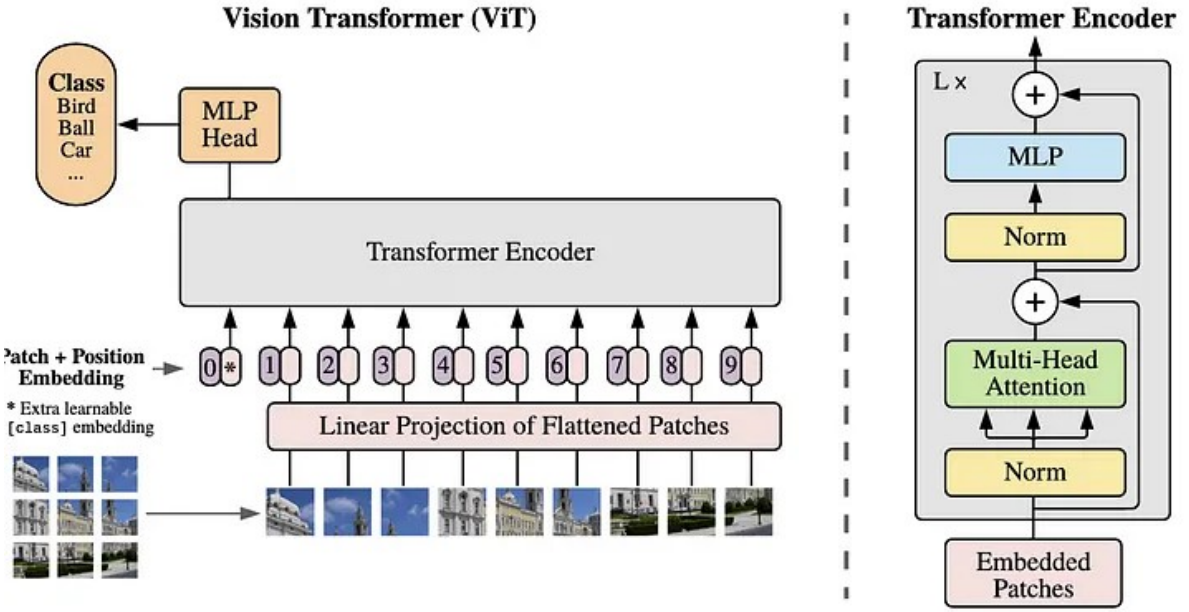
(batch_size, sequence_len, embed_dim)



3 Encoder-only transformers

Encoder-only: a sentiment analysis application would take the input and generate predictions of positive or negative sentiment. This is a *classification task*. No need of casual masking





https://arxiv.org/pdf/2010.11929

Example: classify IMDB reviews as positive or negative

- Database: 50000 reviews labeled as positive or negative
- 25000 for training, 25000 for validation

```
I remember this film,it was the first film i had watched at the cinema the picture was dark in
places i was very nervous it was back in 74/75 my Dad took me my brother & sister to Newbury cinema
in Newbury Berkshire England. I recall the tigers and the lots of snow in the film also the
appearance of Grizzly Adams actor Dan Haggery i think one of the tigers gets shot and dies. If
anyone knows where to find this on DVD etc please let me know.The cinema now has been turned in a
fitness club which is a very big shame as the nearest cinema now is 20 miles away, would love to
hear from others who have seen this film or any other like it.
```
Positive

```
Protocol is an implausible movie whose only saving grace is that it stars Goldie Hawn along with a
good cast of supporting actors. The story revolves around a ditzy cocktail waitress who becomes
famous after inadvertently saving the life of an Arab dignitary. The story goes downhill halfway
through the movie and Goldie's charm just doesn't save this movie. Unless you are a Goldie Hawn fan
don't go out of your way to see this film.
```
Negative

```
DON'T TORTURE A DUCKLING is one of Fulci's earlier (and honestly, in terms of story-line,
better...) films - and although not the typical "bloodbath" that Fulci is known for - this is still
a very unique and enjoyable film.<br /><br />The story surrounds a small town where a series of
child murders are occurring. Some of the colorful characters involved in the investigations -
either as suspects, or those "helping" the investigation (or in some cases both) - include the
towns police force, a small-time reporter, a beautiful and rich ex-drug addict, a young priest and
his mother, An old man who practices witchcraft and his female protégé, a mentally handicapped
townsman, and a deaf/mute little girl. All of these people are interwoven into the plot to create
several twists and turns, until the actual killer is revealed...<br /><br />DON'T TORTURE A
DUCKLING is neither a "classical" giallo or a typical Fulci gore film. Although it does contain
elements of both - it is more of an old-fashioned murder mystery, with darker subject matter and a
few scenes of graphic violence (although nothing nearly as strong as some of Fulci's later works).
This is a well written film with lots of twists that kept me guessing up until the end. Recommended
for giallo/murder-mystery fans, or anyone looking to check out some of Fulci's non-splatter films -
but don't despair, DON'T TORTURE still has more than it's fair share of violence and sleaze. Some
may be put off by the subject of the child killings, and one main female character has a strange
habit of hitting on very young boys, which is also kind of disconcerting - but if that type of
material doesn't bother you, then definitely give this one a look. 8.5/10
```
Positive

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

vocab_size = 20000
maxlen = 200  # keep reviews short for simplicity

(x_train, y_train), (x_val, y_val) = keras.datasets.imdb.load_data(num_words=vocab_size)
print("Number of training samples: ", len(x_train))
print("Number of validation samples: ", len(x_val))

# Pad sequences
x_train = keras.preprocessing.sequence.pad_sequences(x_train, maxlen=maxlen)
x_val = keras.preprocessing.sequence.pad_sequences(x_val, maxlen=maxlen)

idx = np.random.randint(0, len(x_train))

print(x_train[idx])
if (y_train[idx] == 0):
    print("Negative review")
else:
    print("Positive review")

word_index = keras.datasets.imdb.get_word_index()

# Reverse it: index → word
reverse_word_index = {value: key for key, value in word_index.items()}

# Decode function
def decode_review(text):
    return " ".join([reverse_word_index.get(i - 3, "?") for i in text])
print(decode_review(x_train[idx]))
```

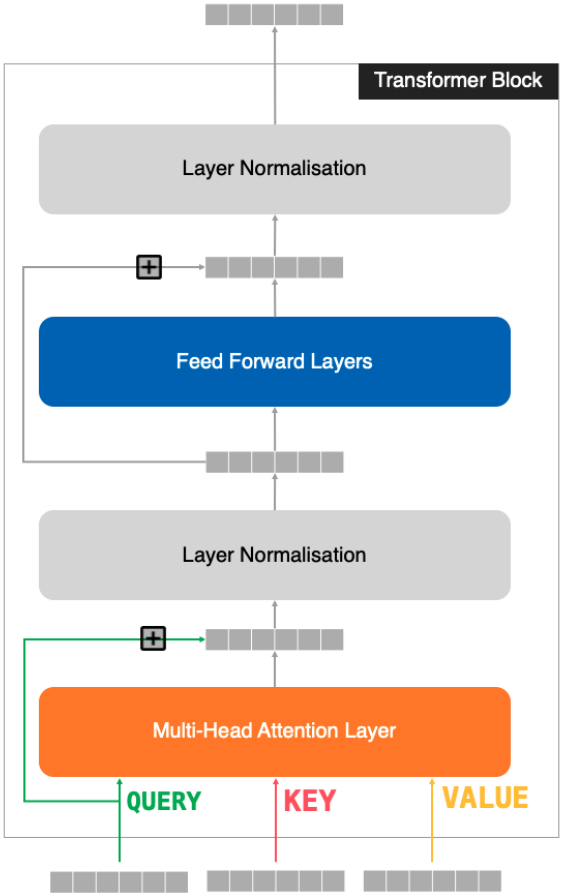
- Text is lowercased
- Punctuation is removed
- Words are tokenized (the most frequent *num_words*)

If longer than *maxlen*, tokens are removed from the **beginning** of the sequence.
If shorter than *maxlen*, zeros are added at the beginning of the sequence

```
-------------------------------------------------------------
Number of training samples:  25000
Number of validation samples:  25000
-------------------------------------------------------------
[    0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     1   745     2     9    35   748   558    29    16 14566    11
   940    14    22    34     4  1325     4   477  2220     9     6    55
  4071    22    19     6    55  3660    65    21    49   480   388     7
   748    40     2     2  4636   914     2  3611   228   152   339   345
    12    16    93    11  1876   268    38     4  1856   152   123    12
    11    32    45  3295   903   440    18     6     2   288  5740   131
    45   290   149    18    49   744  5658   748]
Positive review
? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
```

```
Positive review
? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
? ? ? ? ? ? ? ? richard ? is an animation god he was hampered in directing this film by the producer the final pro
duct is a very uneven film with a very convoluted story but some amazing moments of animation like ? ? greedy joe
? repetitive music doesn't help either it was made in wide screen so the vhs doesn't show it in all it's glory le
t's hope for a ? dvd someday still it's worth watching for some eye popping animation
```

```python
class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super().__init__()
        self.att = layers.MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
        self.dropout1 = layers.Dropout(rate)
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.ffn = keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim),
        ])
        self.dropout2 = layers.Dropout(rate)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)

    def call(self, inputs):
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output)
        out1 = self.layernorm1(inputs + attn_output)

        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output)
        return self.layernorm2(out1 + ffn_output)
```



ff_dim is the size of the intermediate layer of the dense network

embed_dim is the size of the embedding

```python
class TokenAndPositionEmbedding(layers.Layer):
    def __init__(self, maxlen, vocab_size, embed_dim):
        super().__init__()
        self.token_emb = layers.Embedding(input_dim=vocab_size, output_dim=embed_dim)
        self.pos_emb = layers.Embedding(input_dim=maxlen, output_dim=embed_dim)

    def call(self, x):
        positions = tf.range(start=0, limit=tf.shape(x)[-1], delta=1)
        positions = self.pos_emb(positions)
        x = self.token_emb(x)
        return x + positions
```

These are not sinusoidal positional embeddings.
This is a matrix learned when training the model

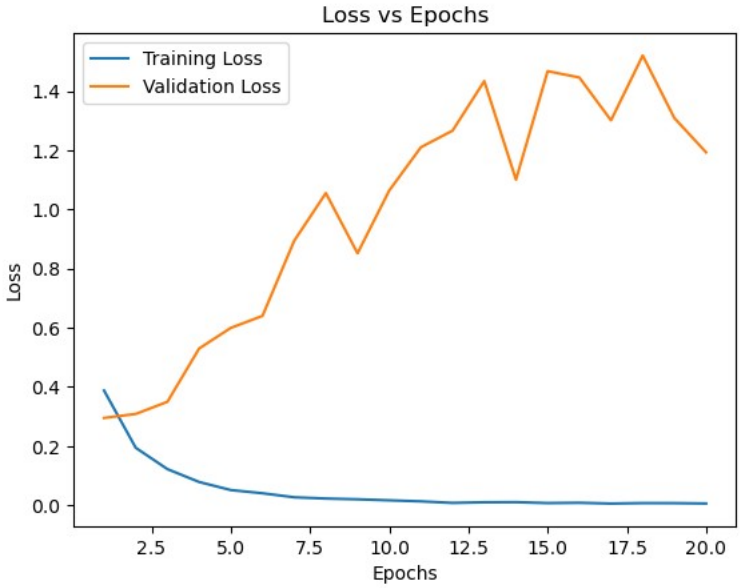
```python
embed_dim = 32
num_heads = 2
ff_dim = 32

inputs = layers.Input(shape=(maxlen,))
embedding_layer = TokenAndPositionEmbedding(maxlen, vocab_size, embed_dim)
x = embedding_layer(inputs)

transformer_block = TransformerBlock(embed_dim, num_heads, ff_dim)
x = transformer_block(x)

x = layers.GlobalAveragePooling1D()(x)
x = layers.Dropout(0.1)(x)
x = layers.Dense(20, activation="relu")(x)
x = layers.Dropout(0.1)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)

model = keras.Model(inputs=inputs, outputs=outputs)
```

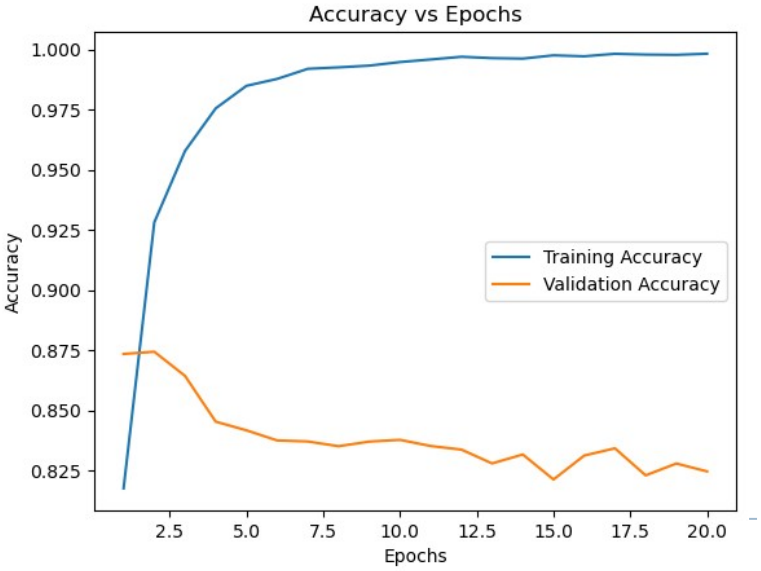
Compresses the sequence into a single vector (taking the average value)

Output size = 1

```python
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
```

```python
history = model.fit(
    x_train, y_train,
    batch_size=32,
    epochs=20,
    validation_data=(x_val, y_val),
)
```





4 Decoder-only transformers

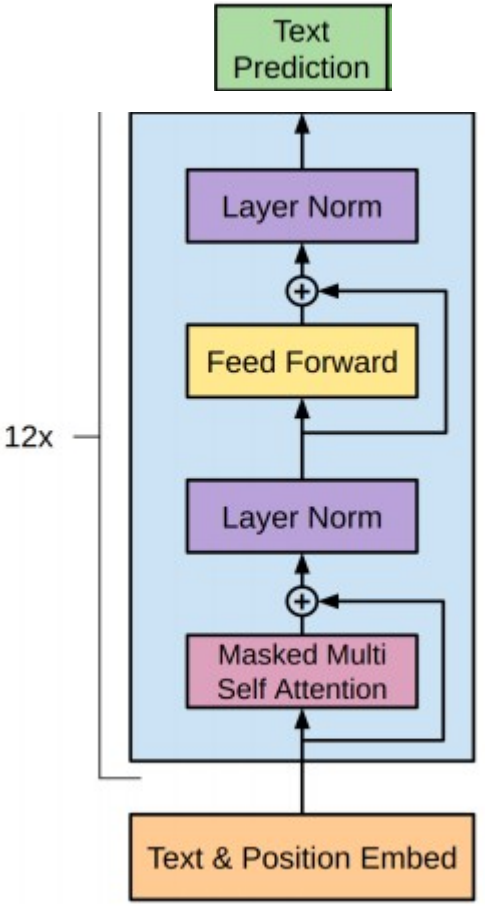
Decoder-only: for *text generation*, the model is trained to predict future values based on previous values:

- It predicts a token in the sequence
- It adds this predicted token to the input
- It predicts the following token

Text: Second Law of Robotics: A robot must obey the orders given it by human beings

Generated training examples

Example #	Input (features)	Correct output (labels)
1	Second law of robotics :	a
2	Second law of robotics : a	robot
3	Second law of robotics : a robot	must
...		



Decoder-only

Example: wine reviews generator

Dataset

https://www.kaggle.com/datasets/zynicide/wine-reviews

{'points': '87', 'title': 'Nicosia 2013 Vulkà Bianco (Etna)', 'description': "Aromas include tropical fruit, broom, brimstone and dried herb. The palate isn't overly expressive, offering unripened apple, citrus and dried sage alongside brisk acidity.", 'taster_name': 'Kerin O'Keefe', 'taster_twitter_handle': '@kerinokeefe', 'price': None, 'designation': 'Vulkà Bianco', 'variety': 'White Blend', 'region_1': 'Etna', 'region_2': None, 'province': 'Sicily & Sardinia', 'country': 'Italy', 'winery': 'Nicosia'}

{'points': '87', 'title': 'Quinta dos Avidagos 2011 Avidagos Red (Douro)', 'description': "This is ripe and fruity, a wine that is smooth while still structured. Firm tannins are filled out with juicy red berry fruits and freshened with acidity. It's already drinkable, although it will certainly be better from 2016.", 'taster_name': 'Roger Voss', 'taster_twitter_handle': '@vossroger', 'price': 15, 'designation': 'Avidagos', 'variety': 'Portuguese Red', 'region_1': None, 'region_2': None, 'province': 'Douro', 'country': 'Portugal', 'winery': 'Quinta dos Avidagos'}

Let's see some code....

1. Hyperparameters and dataset

```
VOCAB_SIZE = 10000
MAX_LEN = 80
EMBEDDING_DIM = 256
KEY_DIM = 256
N_HEADS = 2
FEED_FORWARD_DIM = 256
VALIDATION_SPLIT = 0.2
SEED = 42
BATCH_SIZE = 32
EPOCHS = 5
```

```
# Load the full dataset
# Download from https://www.kaggle.com/datasets/zynicide/wine-reviews

with open("/home/leandro/datasets/winemag-data/winemag-data-130k-v2.json") as json_data:
    wine_data = json.load(json_data)

print("Number of wine reviews: ", len(wine_data))
```

```
Number of wine reviews:  129971
```

```
# Filter the dataset
filtered_data = [
    "wine review : "
    + x["country"]
    + " : "
    + x["province"]
    + " : "
    + x["variety"]
    + " : "
    + x["description"]
    for x in wine_data
    if x["country"] is not None
    and x["province"] is not None
    and x["variety"] is not None
    and x["description"] is not None
]
```

```
# Count the recipes
n_wines = len(filtered_data)
print(f"{n_wines} filtered recipes loaded")
```

```
129907 filtered recipes loaded
```

```
example = filtered_data[2]
print(example)
```

```
wine review : US : Oregon : Pinot Gris : Tart and snappy, the flavors of lime flesh and rind dominate. Some green pineapple pokes through, with crisp acidity underscoring the flavors. The wine was all stainless-steel fermented.
```

2. Tokenization and padding

```
# Display an example of a recipe
example_data = text_data[2]
example_data
```

```
'wine review : US : Oregon : Pinot Gris : Tart and snappy , the flavors of lime flesh and ri
nd dominate . Some green pineapple pokes through , with crisp acidity underscoring the flavo
rs . The wine was all stainless - steel fermented . '
```

Punctuation marks are padded (they are treated as words)

```
# Create a vectorisation layer
vectorize_layer = layers.TextVectorization(
    standardize="lower",
    max_tokens=VOCAB_SIZE,
    output_mode="int",
    output_sequence_length=MAX_LEN + 1,
)
```

```
# Display the same example converted to ints
example_tokenised = vectorize_layer(example_data)
print(example_tokenised.numpy())
```

```
[   7   10    2   20    2  150    2   43  410    2  138    5 1008    3
    6   16    9  149 1029    5  680  626    4  104   94  234 6404   84
    3   11   73   30 5781    6   16    4    6    7  439  127  878   14
  796  541    4    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0]
```

```
# Display some token:word mappings
for i, word in enumerate(vocab[:20]):
    print(f"{i}: {word}")
```

```
0:
1: [UNK]
2: :
3: ,
4: .
5: and
6: the
7: wine
8: a
9: of
10: review
11: with
12: this
13: is
14: -
15: it
16: flavors
17: in
18: '
19: to
```

3. Create training set and masking

```python
# Create the training set of recipes and the same text shifted by one word
def prepare_inputs(text):
    text = tf.expand_dims(text, -1)
    tokenized_sentences = vectorize_layer(text)
    x = tokenized_sentences[:, :-1]
    y = tokenized_sentences[:, 1:]
    return x, y

train_ds = text_ds.map(prepare_inputs)
```

```
<tf.Tensor: shape=(80,), dtype=int64, numpy=
array([   7,   10,    2,   42,    2,   65,    2,   65,   14,   63,   24,
         27,    2,    6, 1480,  853,   13,    9,    8,  315,    3,  222,
         72,    7,    4, 1818,   87,   59,   70,    3,  104,  126,    5,
          8,  166,  413,  249, 1266,   17,    4,   11,   54,   72,  107,
          5,   93,  269,    3,   12,   13,    8,    7,  177,   19,   35,
         66,    4,  802,    4,    0,    0,    0,    0,    0,    0,    0,
          0,    0,    0,    0,    0,    0,    0,    0,    0,    0,    0,
          0,    0,    0])>
```

Training input

Training output

```
<tf.Tensor: shape=(80,), dtype=int64, numpy=
array([  10,    2,   42,    2,   65,    2,   65,   14,   63,   24,   27,
          2,    6, 1480,  853,   13,    9,    8,  315,    3,  222,   72,
          7,    4, 1818,   87,   59,   70,    3,  104,  126,    5,    8,
        166,  413,  249, 1266,   17,    4,   11,   54,   72,  107,    5,
         93,  269,    3,   12,   13,    8,    7,  177,   19,   35,   66,
          4,  802,    4,    0,    0,    0,    0,    0,    0,    0,    0,
          0,    0,    0,    0,    0,    0,    0,    0,    0,    0,    0,
          0,    0,    0])>
```

```
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]], dtype=int32)
```

Causal masking: we don't want future words to be used in the prediction

This is an example: actual matrix is *(MAX_LEN, MAX_LEN) = (80,80)*

4. The model

```
inputs = layers.Input(shape=(None,), dtype=tf.int32)
x = TokenAndPositionEmbedding(MAX_LEN, VOCAB_SIZE, EMBEDDING_DIM)(inputs)
x, attention_scores = TransformerBlock(
    N_HEADS, KEY_DIM, EMBEDDING_DIM, FEED_FORWARD_DIM
)(x)
outputs = layers.Dense(VOCAB_SIZE, activation="softmax")(x)
gpt = models.Model(inputs=inputs, outputs=[outputs, attention_scores])
gpt.compile("adam", loss=[losses.SparseCategoricalCrossentropy(), None])
```

Positional embedding is not sinusoidal
EMBEDDING_DIM = 256
Token embedding: 10000*256
Position embedding: 80*256

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, None)	0
token_and_position_embedding (TokenAndPositionEmbedding)	(None, None, 256)	2,580,480
transformer_block (TransformerBlock)	[(None, None, 256), (None, 2, None, None)]	658,688
dense_2 (Dense)	(None, None, 10000)	2,570,000

Total params: 5,809,168 (22.16 MB)

The model returns a vector with 10000 coefficients (probabilities of each word in the dictionary)

5. Training

```
generated text:
wine review : france : champagne : champagne blend : this is an attractive , fruity and fruity wine that is packed with lightly van
illa and a taut body . ready to drink it now . it is a ripe wine , full in feel , lively and attractive acidity . of orange fruits
and plenty of acidity end that pairs beautifully by white fruits .
```

```
generated text:
wine review : us : new york : riesling : while exceptionally ripe peach and yellow from the end of this dry riesling is penetrating
, concentrated enough on the palate . intensely viscous grapefruit and apricot burst out on the midpalate . full - bodied finish le
nds a lift .
```

```
generated text:
wine review : us : california : pinot noir : blended from russian river valley , this is a crisp , green valley pinot noir and show
s a deep nose of black cherry and vanilla aromas , a flash of cranberry on the palate . cranberry acidity carries with a chalky lay
er of white flowers , cedar and wet gravel minerality .
```

```
generated text:
wine review : austria : weinviertel : grüner veltliner : a ripe baked apple scent this fruity wine , juicy and very fresh for white
peach right on the palate . it does have a sweet grapefruit edge while it ' s a wine that has freshness and a fresh , easygoing tex
ture that needs to age .
```

```
generated text:
wine review : spain : catalonia : macabeo : berry , earth , leather and mild , briny aromas are all on the palate . a flush , ultra
- healthy , mildly green flavor profile offers tomato and stone fruit flavors right in scope and through a lasting lime finish . gr
eat value and young price .
```

NUM_EPOCHS = 5

It generates a review at the end of each epoch

Lots and lots of warnings to ignore!!

6. Using the model to generate reviews

```
info = text_generator.generate(
    "wine review : us", max_tokens=80, temperature=1.0
)
```

```
generated text:
wine review : us : washington : bordeaux - style red blend : the [UNK] from a biodynamic vineyar
d , this blend of top chardonnay , grenache , merlot , cabernet sauvignon , and it brings aromas
of raspberries and sweet jammy green - fruit . it ' s lightly sweet and seems to be quite overwr
ought , with a creamy feel and immediate enjoyment .
```

```
info = text_generator.generate(
    "wine review : italy", max_tokens=80, temperature=0.5
)
```

```
generated text:
wine review : italy : tuscany : red blend : this blend of sangiovese and cabernet sauvignon is a
n easy and informal wine with bright cherry and blackberry flavors . the wine is soft and easy t
o drink .
```

```
info = text_generator.generate(
    "wine review : germany", max_tokens=80, temperature=0.5
)
print_probs(info, vocab)
```

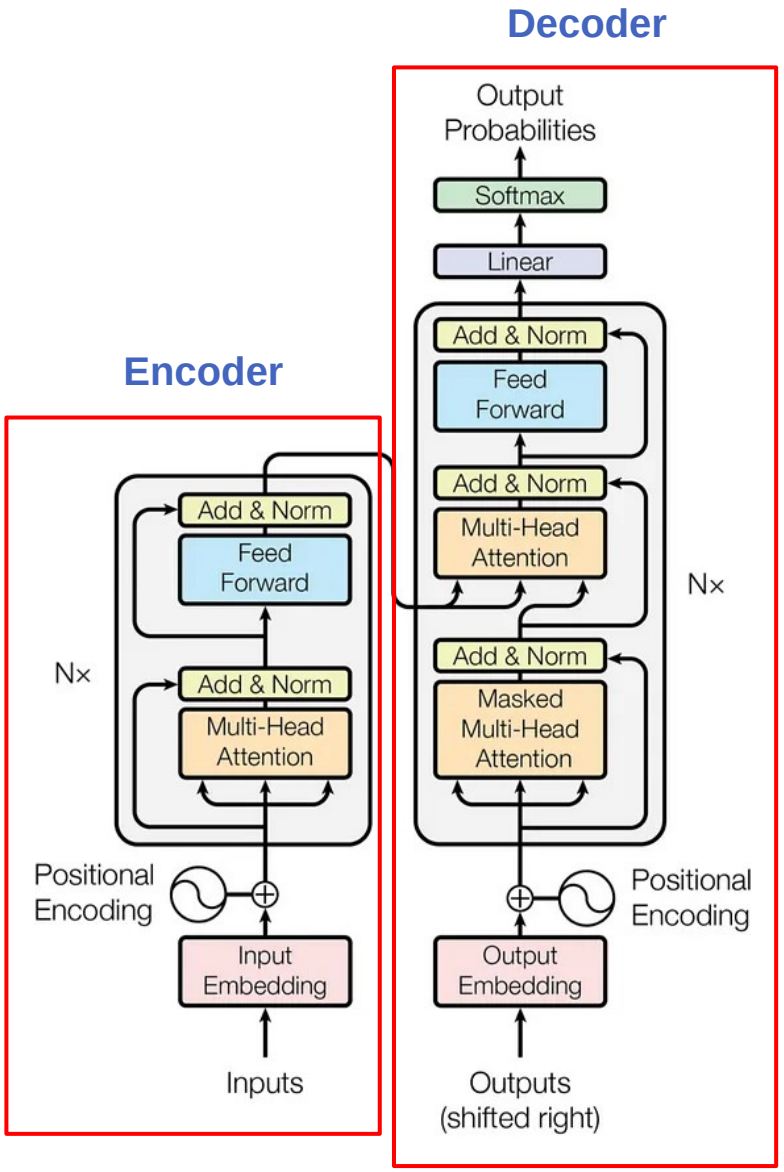
```
generated text:
wine review : germany : rheingau : riesling : a wisp of smoke and mineral mark the nose of this
feather - light auslese . it ' s a bit lean , with tart tangerine and grapefruit flavors . it '
s a bit lean and taut , with a slightly chalky , dry finish .
```

Temperature sets how random is the text generation

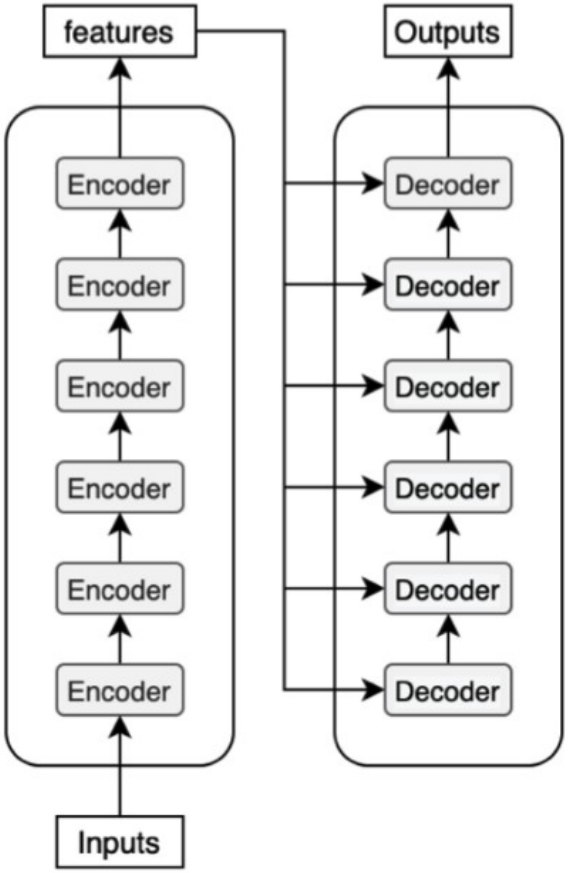
If temperature is low, the generation is deterministic (the same seed generates the same review)

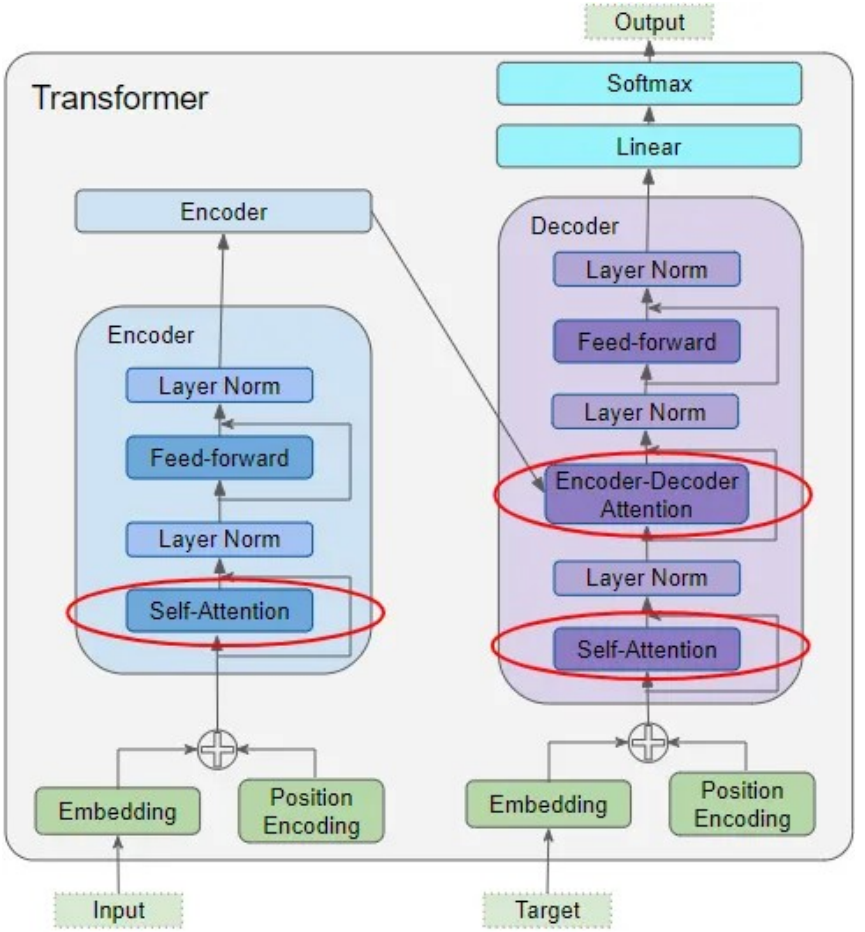
```
wine review : germany : rheingau : riesling : a
hint:           30.899999618530273%
whiff:          12.579999923706055%
bit:    12.430000305175781%
wisp:           7.019999980926514%
touch:          4.349999904632568%
--------

wine review : germany : rheingau : riesling : a wisp
of:     100.0%
and:    0.0%
to:     0.0%
,:      0.0%
that:           0.0%
--------

wine review : germany : rheingau : riesling : a wisp of
smoke:          97.58999633789062%
a:      1.1699999570846558%
honey:          0.4699999988079071%
wax:    0.11999999731779099%
petrol:         0.10999999940395355%
--------

wine review : germany : rheingau : riesling : a wisp of smoke
and:    61.27000045776367%
lends:          20.600000381469727%
notes:          5.559999942779541%
,:      3.5199999809265137%
tones:          1.1699999570846558%
--------

wine review : germany : rheingau : riesling : a wisp of smoke and
mineral:        23.940000534057617%
smoke:          20.93000030517578%
nut:    10.960000038146973%
a:      6.320000171661377%
crushed:        6.300000190734863%
--------
```

5 Encoder-decoder architecture



- Typical application: translation
- Nx means a stack, both in the encoder and the decoder
- All encoder blocks are identical
- Encoder: no masking on the attention layer to capture all cross-dependencies between words, independently of the order
- All decoder blocks are identical
- Decoder: initial attention layer is *self-referential* (K, Q, V from the same input) and masked
- Decoder: subsequent attention layer is *cross-referential*. Q from the decoder, K and V from the encoder (the decoder gets the encoder's representation of the sentence to be translated)

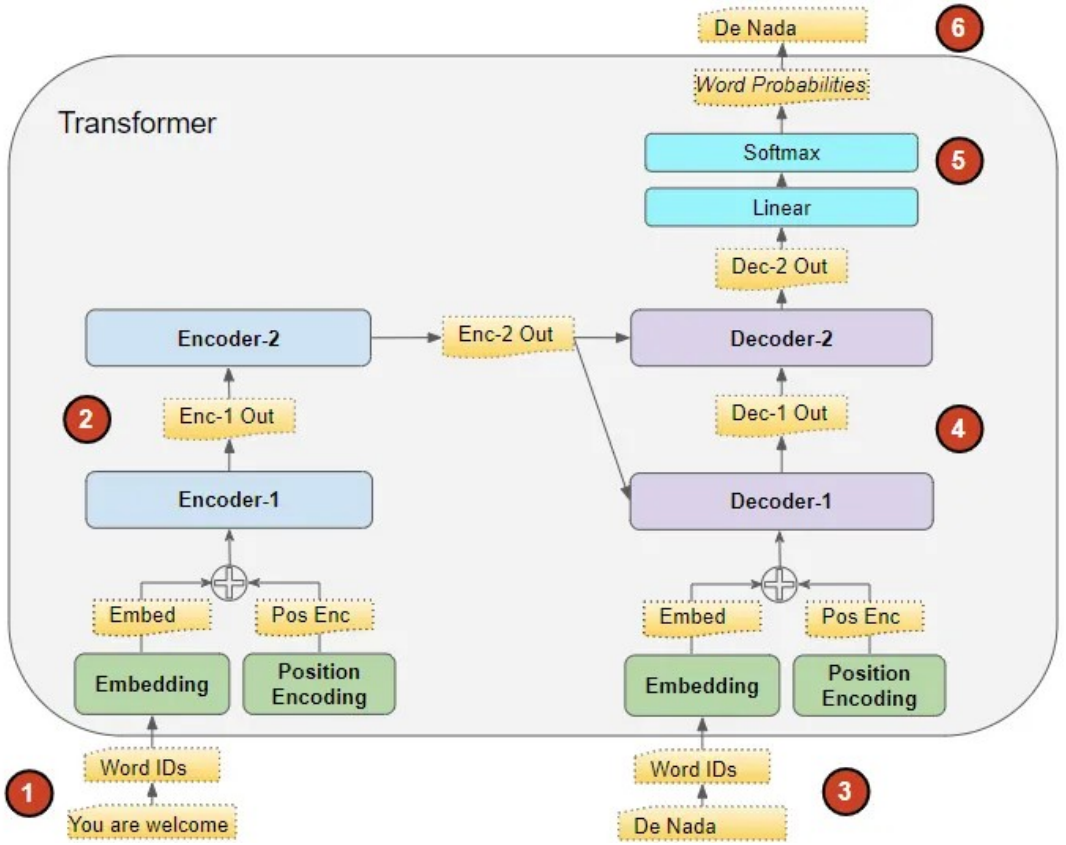




- Encoder self-attention: gets input-embedding + position encoding and produces an encoded representation including attention scores.
- Decoder self-attention: the same, using casual masking
- Encoder-decoder attention: gets a representation of the input sequence (with attention) and the target sequence (with attention). Produces a representation with the attention scores for each target sequence word that captures the influence of the attention scores from the input sequence as well.

Training the transformer

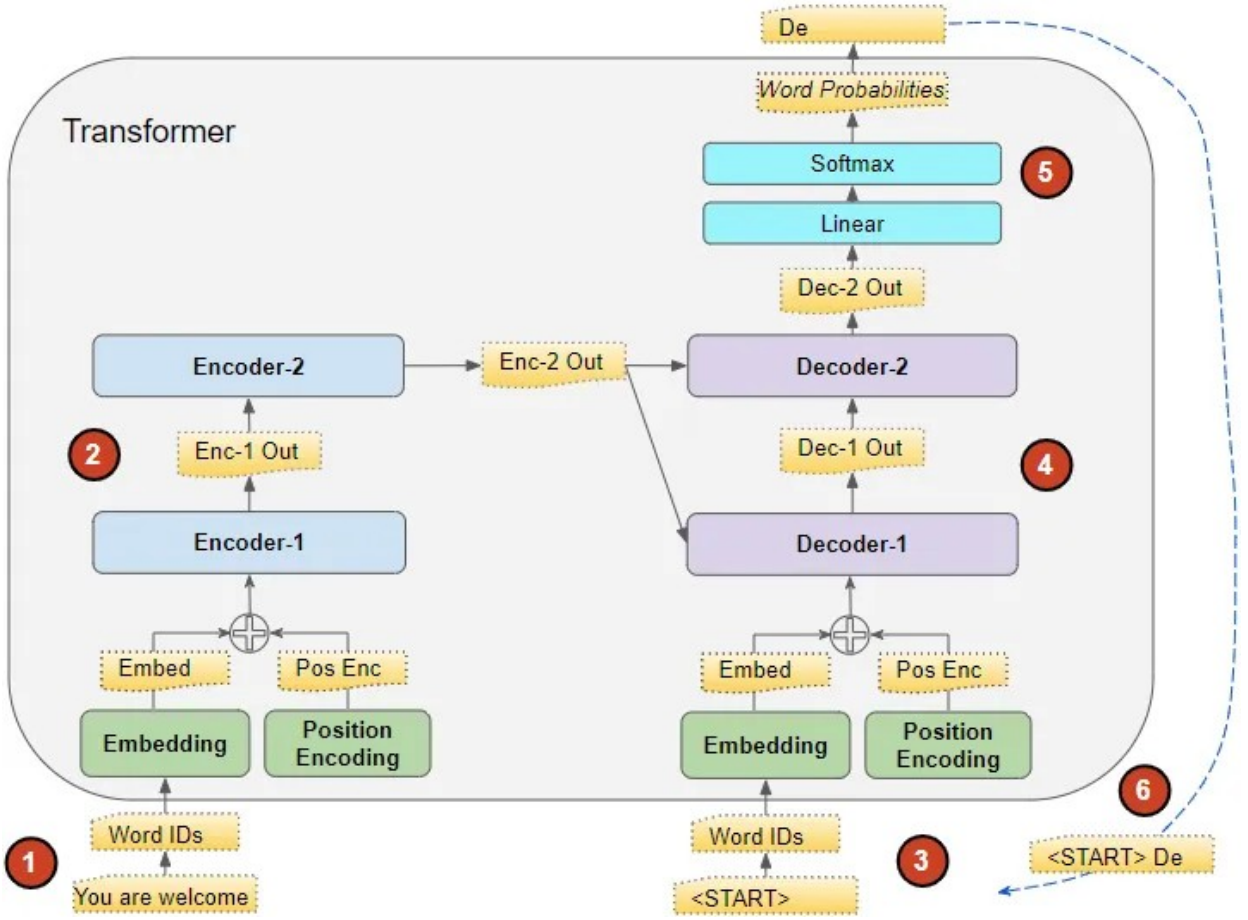
- Source: “you are welcome”
- Target: “de nada”



1) The input sequence is put between <START> and <END> and fed into the encoder
2) The encoder produces an encoded representation of the input
3) The target is put between <START> and <END>, and fed into the decoder
4) The decoder processes this with the encoder’s output to produce an encoded representation of the target
5) This is converted into word probabilities
6) The output is compared against the target from the training data. The result of the loss fuction is used to generate gradients to train the transformer using back-propagation

Inference

- We have only the input source
- We generate the output in a loop and feed input+output to the generator until an end-of-sequence token is produced



1) The input sequence is put between <START> and <END> and fed into the encoder
2) The encoder produces an encoded representation of the input
3) The target is only a <START>, and fed into the decoder
4) The decoder processes this with the encoder's output to produce an encoded representation of the target
5) This is converted into word probabilities
6) Go back to step #3 until the decoder generates an <END> token

Note: steps #1 and #2 are executed only once

A toy example: an English to Spanish translator

1. Hyperparameters and dataset

```python
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
    # Reduces the number of warnings

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers


# Hyperparameters
# =====================================

NUM_EPOCHS = 200
MAX_VOCAB = 100
MAX_LEN = 10
```

```python
# Toy dataset
# =====================================

english_sentences = [
    "hello",
    "how are you",
    "i love coffee"
]

spanish_sentences = [
    "<start> hola <end>",
    "<start> como estás <end>",
    "<start> me gusta el café <end>"
]

# Tokenization
# =====================================

src_vec = layers.TextVectorization(max_tokens=MAX_VOCAB, output_mode="int", output_sequence_length=MAX_LEN)
tgt_vec = layers.TextVectorization(max_tokens=MAX_VOCAB, output_mode="int", output_sequence_length=MAX_LEN)

src_vec.adapt(english_sentences)
tgt_vec.adapt(spanish_sentences)

print("Source vocabulary:", src_vec.get_vocabulary())
print("Target vocabulary:", tgt_vec.get_vocabulary())

encoder_input = src_vec(english_sentences)
target_seq = tgt_vec(spanish_sentences)

decoder_input = target_seq[:, :-1]
decoder_target = target_seq[:, 1:]

print(decoder_input)
print(decoder_target)
```

```
Source vocabulary: ['', '[UNK]', np.str_('you'), np.str_('love'), np.s
tr_('i'), np.str_('how'), np.str_('hello'), np.str_('coffee'), np.str_
('are')]
Target vocabulary: ['', '[UNK]', np.str_('start'), np.str_('end'), np.
str_('me'), np.str_('hola'), np.str_('gusta'), np.str_('estás'), np.st
r_('el'), np.str_('como'), np.str_('café')]
tf.Tensor(
[[ 2  5  3  0  0  0  0  0  0]
 [ 2  9  7  3  0  0  0  0  0]
 [ 2  4  6  8 10  3  0  0  0]], shape=(3, 9), dtype=int64)
tf.Tensor(
[[ 5  3  0  0  0  0  0  0  0]
 [ 9  7  3  0  0  0  0  0  0]
 [ 4  6  8 10  3  0  0  0  0]], shape=(3, 9), dtype=int64)
```

decoder_target is displaced by one word

2. Masks layers and embeddings

```python
class PaddingMask(layers.Layer):
    def call(self, x):
        mask = tf.cast(tf.not_equal(x, 0), tf.float32)
        return mask[:, tf.newaxis, :]

class LookAheadMask(layers.Layer):
    def call(self, x):
        seq_len = tf.shape(x)[1]
        mask = 1 - tf.linalg.band_part(tf.ones((seq_len, seq_len)), -1, 0)
        return mask[tf.newaxis, :, :]


# Positional Embedding
# ====================================

class PositionalEmbedding(layers.Layer):
    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.embedding = layers.Embedding(vocab_size, d_model)
        self.position = layers.Embedding(MAX_LEN, d_model)

    def call(self, x):
        length = tf.shape(x)[1]
        positions = tf.range(start=0, limit=length, delta=1)
        return self.embedding(x) + self.position(positions)
```

Applies a mask so padding is not considered as part of the sentences

Casual masking

Combines token and positional embeddings

3. Encoder and decoder

For each input word, it gives a vector with full sentence context to be used in the decoder

```python
# Encoder
# ====================================

class Encoder(layers.Layer):
    def __init__(self, vocab_size, d_model, num_heads):
        super().__init__()
        self.embed = PositionalEmbedding(vocab_size, d_model)
        self.mask = PaddingMask()
        self.mha = layers.MultiHeadAttention(num_heads=num_heads, key_dim=d_model)
        self.norm = layers.LayerNormalization()

    def call(self, x):
        x_emb = self.embed(x)
        mask = self.mask(x)
        attn = self.mha(x_emb, x_emb, attention_mask=mask)
        return self.norm(x_emb + attn)
```

```python
# Decoder
# ====================================

class Decoder(layers.Layer):
    def __init__(self, vocab_size, d_model, num_heads, dff):
        super().__init__()
        self.embed = PositionalEmbedding(vocab_size, d_model)

        self.look_ahead = LookAheadMask()
        self.self_attn = layers.MultiHeadAttention(num_heads=num_heads, key_dim=d_model)
        self.norm1 = layers.LayerNormalization()

        self.cross_attn = layers.MultiHeadAttention(num_heads=num_heads, key_dim=d_model)
        self.norm2 = layers.LayerNormalization()

        self.ffn = keras.Sequential([
            layers.Dense(dff, activation="relu"),
            layers.Dense(d_model),
        ])
        self.norm3 = layers.LayerNormalization()


    def call(self, x, enc_output):
        x_emb = self.embed(x)

        la_mask = self.look_ahead(x)
        attn1 = self.self_attn(x_emb, x_emb, attention_mask=la_mask)
        x = self.norm1(x_emb + attn1)

        attn2 = self.cross_attn(x, enc_output)
        x = self.norm2(x + attn2)

        ffn_out = self.ffn(x)
        return self.norm3(x + ffn_out)
```

Embedding tokens + positions → represents generated words with order information

Masked self-attention → looks only at past generated tokens (no peeking ahead)

Cross-attention → attends to the encoder output to align with the input sentence

Feed-forward + normalization → refines the representation

4. The full model

```
# Full Model
# ====================================

def build_model(vocab_size, d_model=64, num_heads=2, dff=128):
    enc_inputs = keras.Input(shape=(None,), name="encoder_inputs")
    dec_inputs = keras.Input(shape=(None,), name="decoder_inputs")

    encoder = Encoder(vocab_size, d_model, num_heads)
    decoder = Decoder(vocab_size, d_model, num_heads, dff)

    enc_out = encoder(enc_inputs)
    dec_out = decoder(dec_inputs, enc_out)

    outputs = layers.Dense(vocab_size, activation="softmax")(dec_out)

    return keras.Model([enc_inputs, dec_inputs], outputs)

model = build_model(MAX_VOCAB)

# Loss
# ====================================

loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(reduction="none")

def masked_loss(y_true, y_pred):
    loss = loss_fn(y_true, y_pred)
    mask = tf.cast(tf.not_equal(y_true, 0), tf.float32)
    loss *= mask
    return tf.reduce_sum(loss) / tf.reduce_sum(mask)

model.compile(optimizer="adam", loss=masked_loss)

model.summary()
```

Each word returns a vector of length *vocab_size* (probability of each possible word)

Layer (type)	Output Shape	Param #	Connected to
encoder_inputs (InputLayer)	(None, None)	0	-
decoder_inputs (InputLayer)	(None, None)	0	-
encoder (Encoder)	(None, None, 64)	40,384	encoder_inputs[0…
decoder (Decoder)	(None, None, 64)	90,432	decoder_inputs[0… encoder[0][0]
dense_2 (Dense)	(None, None, 100)	6,500	decoder[0][0]

Total params: 137,316 (536.39 KB)

Trainable params: 137,316 (536.39 KB)

5. Training and inference

```
# Train
# ====================================

model.fit(
    [encoder_input, decoder_input],
    decoder_target,
    epochs=NUM_EPOCHS,
)
```

```
# Inference
# ====================================

def translate(sentence):
    enc_input = src_vec([sentence])

    start_token = tgt_vec(["<start>"])[0][0]
    end_token = tgt_vec(["<end>"])[0][0]

    output = tf.expand_dims([start_token], 0)

    for _ in range(MAX_LEN):
        preds = model([enc_input, output])
        next_token = tf.argmax(preds[:, -1, :], axis=-1).numpy()[0]

        output = tf.concat([output, [[next_token]]], axis=1)

        if next_token == end_token:
            break

    tokens = output.numpy()[0]
    words = [tgt_vec.get_vocabulary()[i] for i in tokens]

    return " ".join(words)

print(translate("hello"))
print(translate("how are you"))
print(translate("i love coffee"))
```

Tokenization removes punctuation marks

```
start hola end
start como estás end
start me gusta el café end
```

A full example: English to German translation

Hyperparameters

```
BATCH_SIZE = 64
MAX_LEN = 40
VOCAB_SIZE = 15000
EMBED_DIM = 256
NUM_HEADS = 8
FF_DIM = 512
EPOCHS = 25
```

Similar to the toy example, with some **key differences**

Dataset

Subset of multi30k corpus:

- Training: 29000 English and German sentences
- Validation: 1014 English and German sentences

Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.
Mehrere Männer mit Schutzhelmen bedienen ein Antriebsradsystem.
Ein kleines Mädchen klettert in ein Spielhaus aus Holz.
Ein Mann in einem blauen Hemd steht auf einer Leiter und putzt ein Fenster.
Zwei Männer stehen am Herd und bereiten Essen zu.

Two young, White males are outside near many bushes.
Several men in hard hats are operating a giant pulley system.
A little girl climbing into a wooden playhouse.
A man in a blue shirt is standing on a ladder cleaning a window.
Two men are at the stove preparing food.

```
def standardize(text):
    text = tf.strings.lower(text)
    text = tf.strings.regex_replace(text, r"[^\w\s]", "")
    return text

src_vectorizer = layers.TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_sequence_length=MAX_LEN,
    standardize=standardize,
)

tgt_vectorizer = layers.TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_sequence_length=MAX_LEN + 1,
    standardize=standardize,
)
```

Positional encoding

```python
def positional_encoding(length, depth):
    positions = tf.cast(tf.range(length)[:, None], tf.float32)
    depth = depth // 2

    i = tf.cast(tf.range(depth)[None, :], tf.float32)
    angle_rates = 1.0 / (10000.0 ** (i / depth))

    angle_rads = positions * angle_rates

    return tf.concat(
        [tf.sin(angle_rads), tf.cos(angle_rads)],
        axis=-1
    )
```

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

The model

Model: "transformer"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 40, 256)	3,840,000
embedding_1 (Embedding)	(None, 40, 256)	3,840,000
encoder (Encoder)	?	2,367,488
decoder (Decoder)	?	4,471,552
dense_4 (Dense)	(None, 40, 15000)	3,855,000

Total params: 55,122,122 (210.27 MB)
Trainable params: 18,374,040 (70.09 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 36,748,082 (140.18 MB)

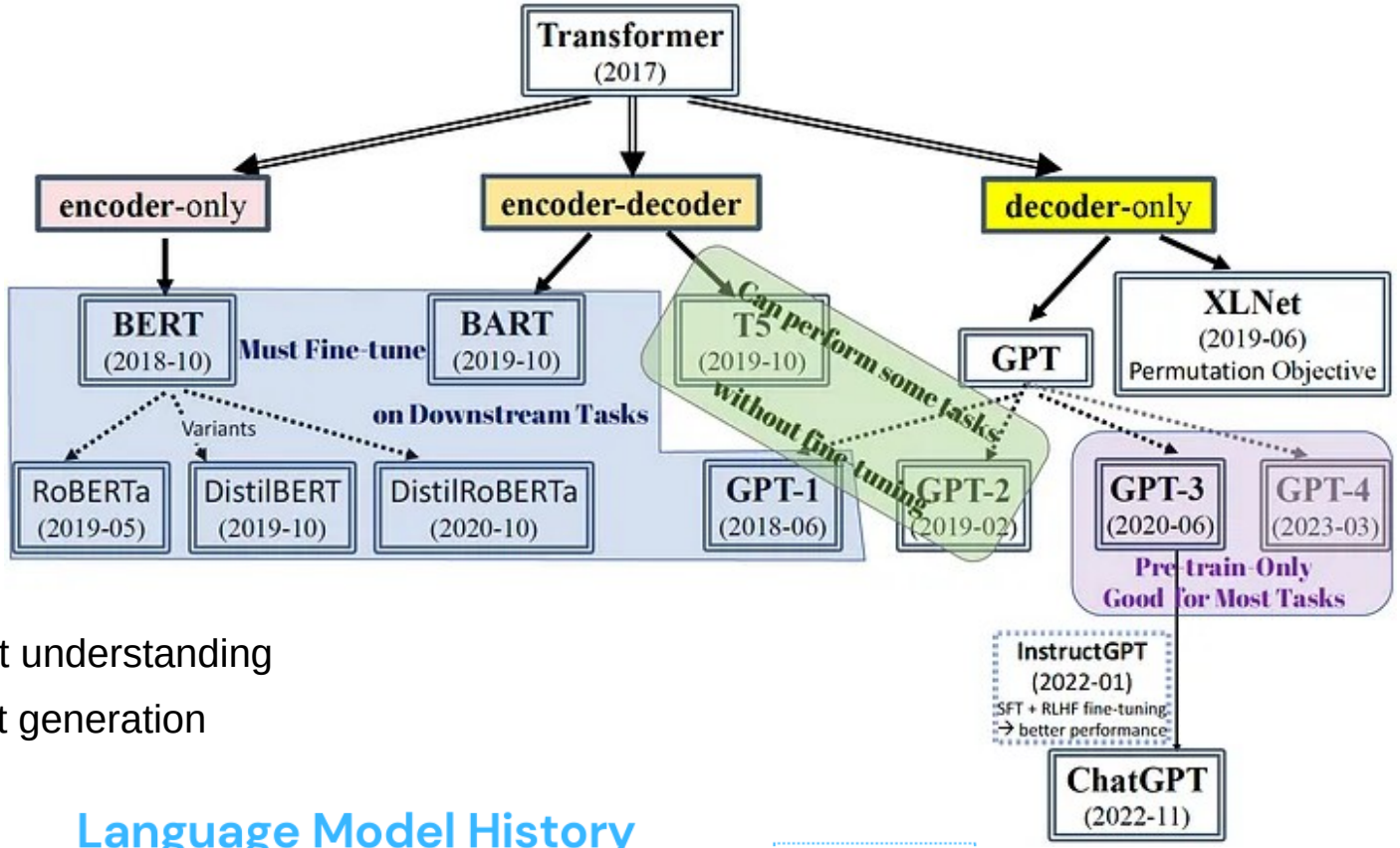
```python
model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
)
```

```
Epoch 1/25
454/454 ━━━━━━━━━━━━━━━━━━━━ 57s 72ms/step - accuracy: 0.7031 - loss: 3.0982 - val_accuracy: 0.7405 - val_loss: 1.7627
Epoch 2/25
454/454 ━━━━━━━━━━━━━━━━━━━━ 11s 23ms/step - accuracy: 0.7563 - loss: 1.5974 - val_accuracy: 0.7631 - val_loss: 1.4908
...
Epoch 24/25
454/454 ━━━━━━━━━━━━━━━━━━━━ 11s 24ms/step - accuracy: 0.8855 - loss: 0.5465 - val_accuracy: 0.8704 - val_loss: 0.7792
Epoch 25/25
454/454 ━━━━━━━━━━━━━━━━━━━━ 11s 24ms/step - accuracy: 0.8881 - loss: 0.5294 - val_accuracy: 0.8698 - val_loss: 0.7695
```

```python
print(translate("zwei männer stehen in der küche"))
```

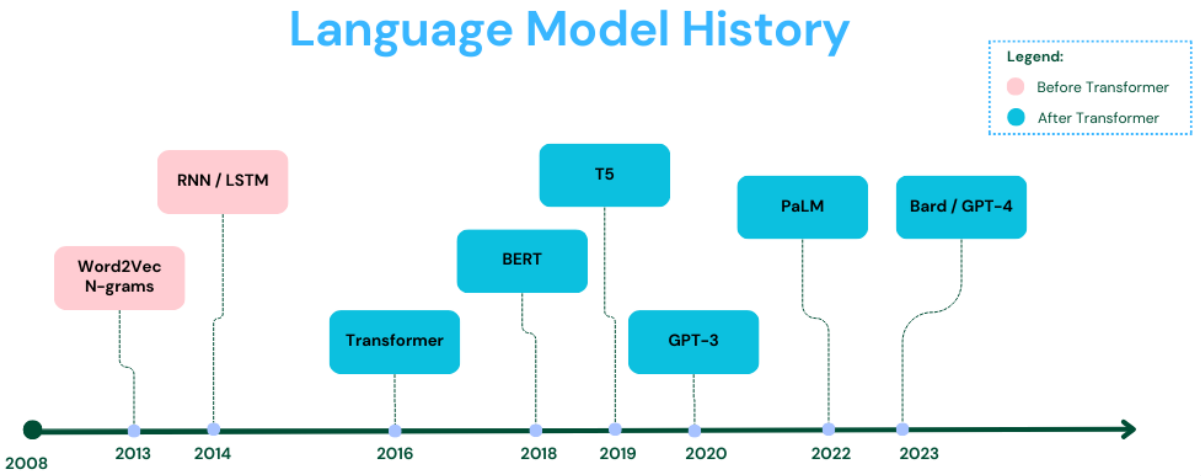
```
start two men are standing in the kitchen end
```

6 Examples of transformers

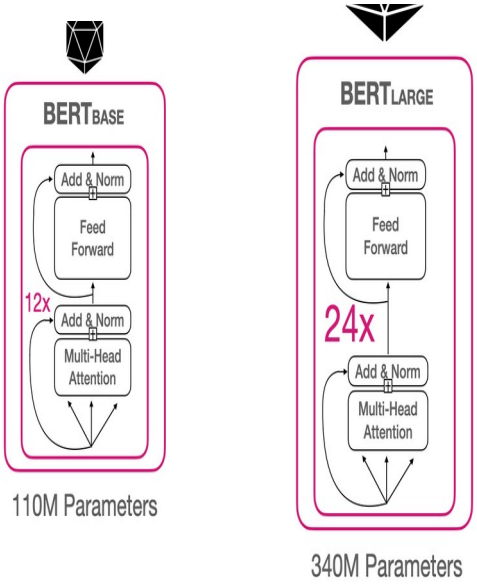


Encoder stack: text understanding

Decoder stack: text generation



BERT (Bidirectional Encoder Representations from Transformers)



Training:

- MLM (Masked Language Model): 15% of words are missing. The model is trained to predict them
- NSP (Next Sentence Prediction): the model is trained to decide if one sentence is the subsequent of another in a document

Using BERT (adding layer to the core model):

- Sentiment analysis
- Find and mark the answer to a question in a text
- Named Entity Recognition: in a text, mark different types of entities (Person, Organization, Date...)

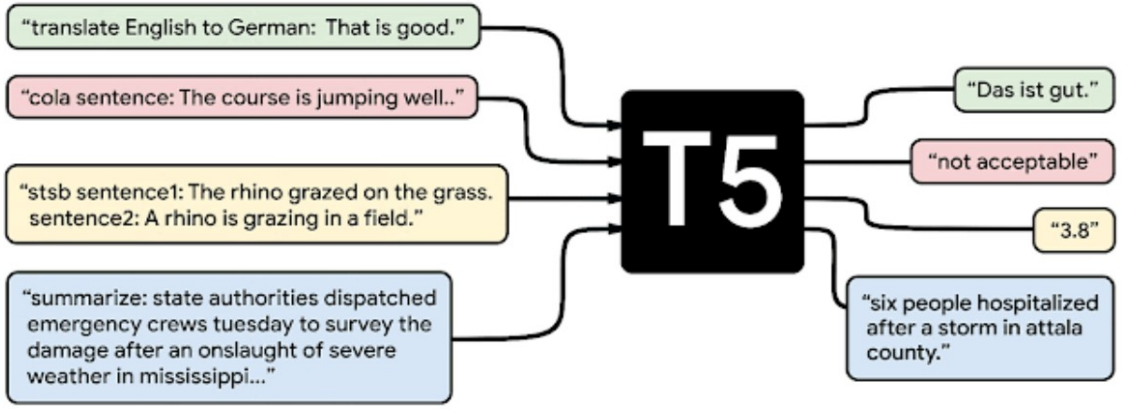
Google, 2018

Encoder-only

345 million parameters

BERT was specifically trained on Wikipedia (~2.5B words) and Google's BooksCorpus (~800M words)

T5 (Text-To-Text Transfer Transformer)



Applications:

- Translation
- cola: involves binary classification for sentence acceptability judgment
- stsb: to calculate semantic text similarity between two sentences
- Summarizing

Name	Total parameters	Encoder parameters	Decoder parameters	n_{layer}	d_{model}	d_{ff}	d_{kv}	n_{head}
Small	76,956,160	35,330,816	41,625,344	6	512	2048	64	8
Base	247,577,856	109,628,544	137,949,312	12	768	3072	64	12
Large	770,567,168	334,939,648	435,627,520	24	1024	4096	64	16
3B	2,884,497,408	1,240,909,824	1,643,587,584	24	1024	16384	128	32
11B	11,340,220,416	4,864,791,552	6,475,428,864	24	1024	65536	128	128

*The encoder and the decoder have the same shape. So for example, the T5-small has 6 layers in the encoder and 6 layers in the decoder.

In the above table,

- n_{layer}: Number of layers in the encoder; also, number of layers in the decoder. They always have the same number of layers.
- n_{head}: Number of attention heads in each attention block.
- d_{model}: Dimension of the embedding vectors.
- d_{ff}: Dimension of the feedforward network within each encoder and decoder layer.
- d_{kv}: Dimension of the key and value vectors used in the self-attention mechanism.

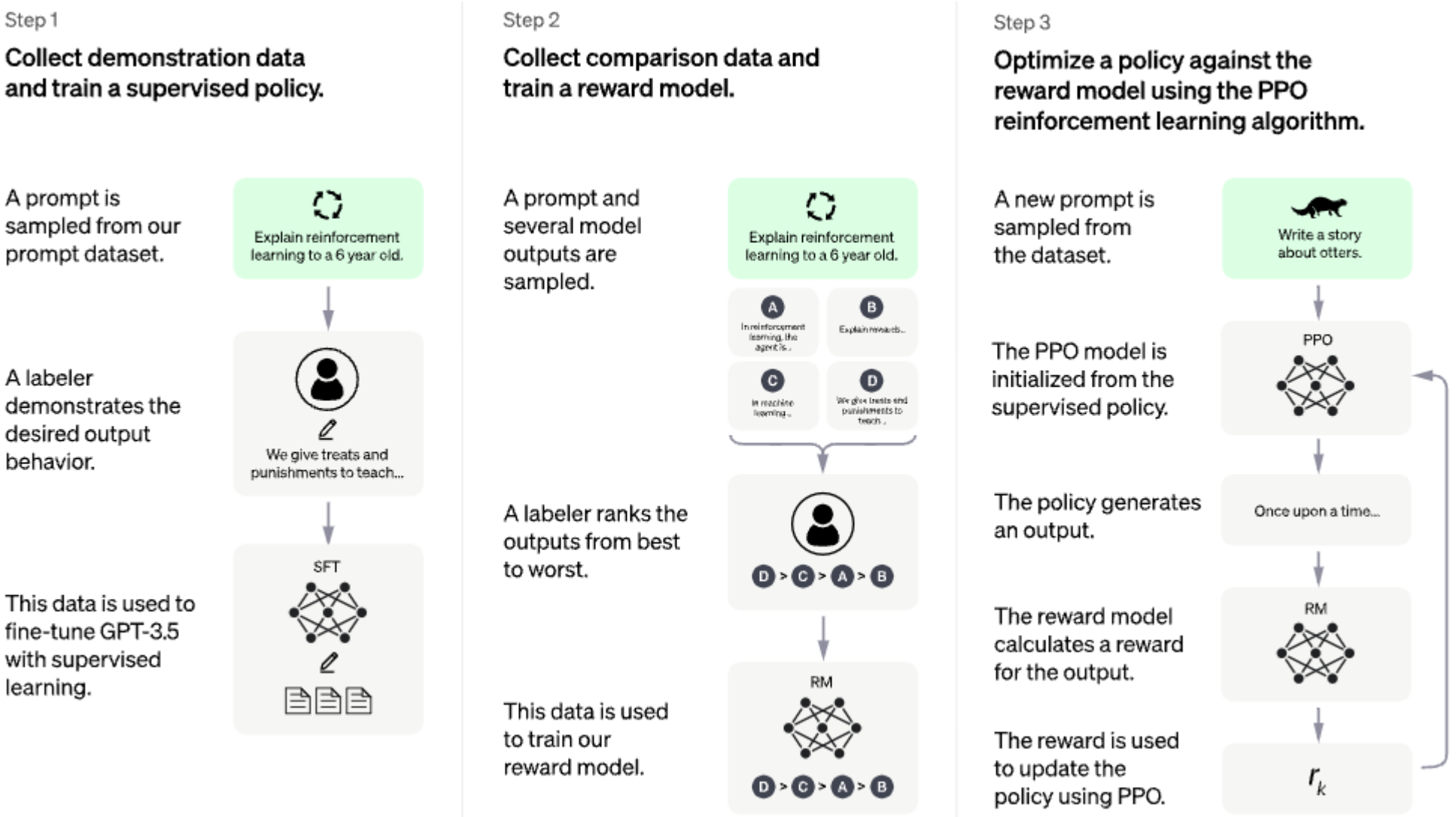
Google, 2020

Encoder-decoder

Trained on 750GB of text
(Colosal Clean Crawled Corpus)

ChatGPT is a a conversational tool fine-tuned from GPT

It uses a technique called Reinforcement Learning from Human Feedback (RLHF) to fine-tune the GPT model



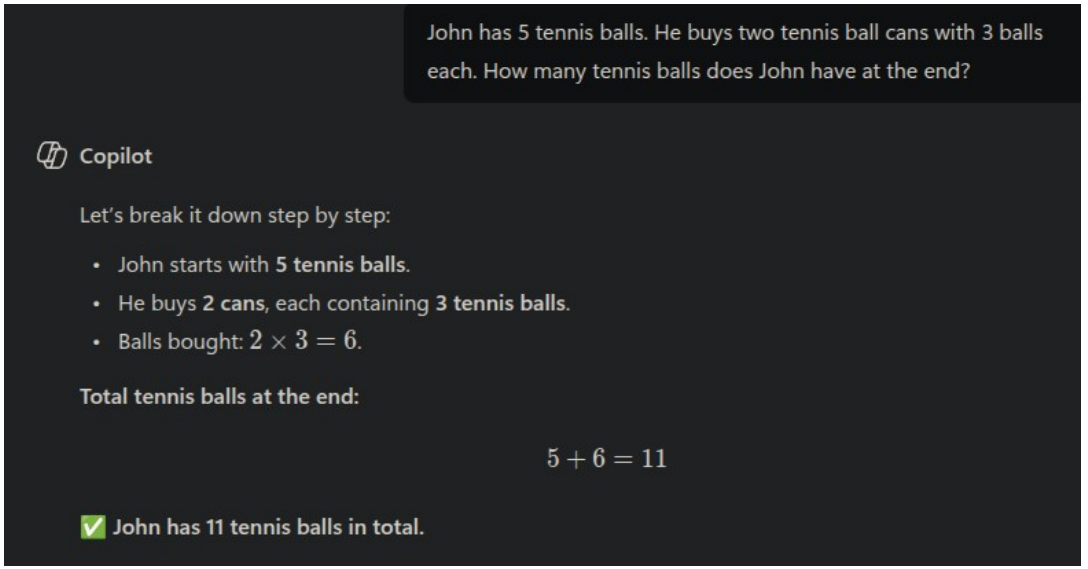
PPO = Proximal Policy Optimization

Deep reasoning models

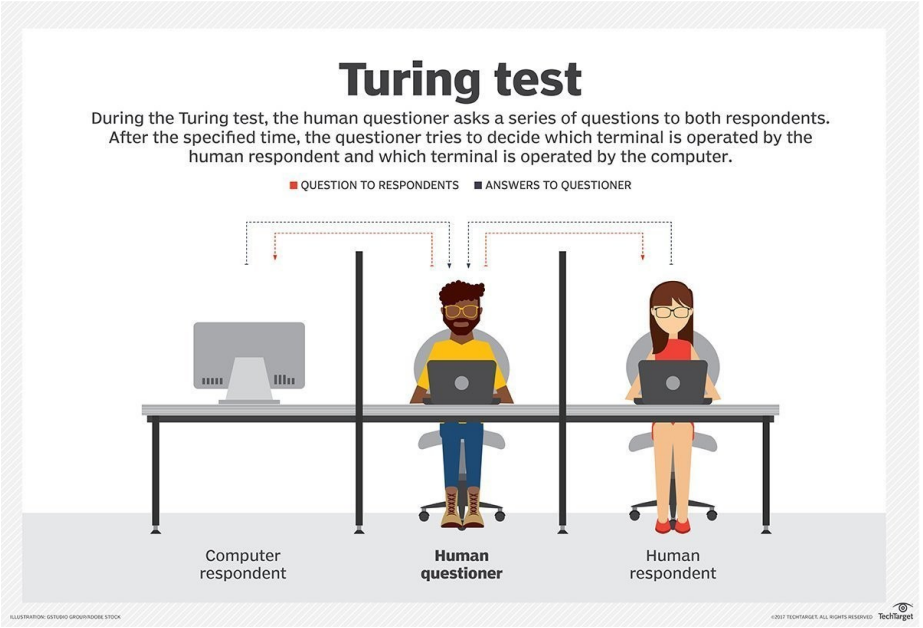
Based on LLMs, with Chain-of-thought (COT) style training: the dataset contains multiple examples of logic propositions, and problems solved step-by-step

The model “learns” to generate the steps that solve a problem, and solve these steps in order

The core of these systems is a transfomer-based LLM



But the models are not intelligent, aren’t they?



Are they stochastic parrots?